
Desarrollo de una Plataforma Web Inteligente
para Nutricionistas
Development of an Intelligent Web Platform for
Nutritionists



Trabajo de Fin de Grado
Curso 2025–2026

Autor

Víctor Martínez Blanco

Director

Alejandro Hernández Cerezo

Pablo Gutiérrez Sánchez

Colaborador

Colaborador 1

Colaborador 2

Doble Grado en Ingeniería Informática y ADE

Facultad de Informática

Universidad Complutense de Madrid

Desarrollo de una Plataforma Web Inteligente para Nutricionistas Development of an Intelligent Web Platform for Nutritionists

Trabajo de Fin de Grado en Ingeniería Informática

Autor

Víctor Martínez Blanco

Director

Alejandro Hernández Cerezo

Pablo Gutiérrez Sánchez

Colaborador

Colaborador 1

Colaborador 2

Convocatoria: Septiembre 2026

Doble Grado en Ingeniería Informática y ADE

Facultad de Informática

Universidad Complutense de Madrid

DIA de MES de AÑO

Dedicatoria

*A Pedro Pablo y Marco Antonio, por crear
TeXiS e iluminar nuestro camino*

Agradecimientos

A Guillermo, por el tiempo empleado en hacer estas plantillas. A Adrián, Enrique y Nacho, por sus comentarios para mejorar lo que hicimos. Y a Narciso, a quien no le ha hecho falta el Anillo Único para coordinarnos a todos.

Resumen

Desarrollo de una Plataforma Web Inteligente para Nutricionistas

Este Trabajo de Fin de Grado presenta el diseño de NutriApp, una plataforma web pensada para centralizar el flujo de trabajo de los nutricionistas profesionales. La idea surgió al observar de cerca cómo trabaja un nutricionista en su día a día: su información estaba repartida entre cuadernos físicos, formularios de Google, Google Calendar, la plataforma de su clínica, notas del móvil y WhatsApp. Si se quería analizar el estado completo de un solo cliente, se tenía que cruzar datos de todas estas herramientas de forma manual; lo cual resulta ineficiente y es probable perder información por el camino.

A partir de este problema real, se propuso diseñar una aplicación web que reuniera todas estas funcionalidades en un único punto de acceso. La plataforma se estructura en dos paneles diferenciados: uno orientado al nutricionista, desde el que puede gestionar clientes, citas, planes nutricionales y notas privadas; y otro orientado al cliente, con una interfaz propia donde consultar su plan, registrar sus hábitos diarios, comunicarse con su nutricionista y hacer seguimiento de su progreso.

Además de sustituir las herramientas dispersas, NutriApp incorpora una funcionalidad innovadora: la creación de dietas personalizadas para cada cliente. Esta herramienta tan específica solo es posible de llevar a cabo con éxito cuando gran parte de la información relativa al usuario está conectada, siendo posible acceder a sus alérgenos, intolerancias, objetivos nutricionales, preferencias y hábitos entre otros. Además, al mismo tiempo, es de gran relevancia las normas, preferencias y restricciones que imponga su nutricionista al programa, ya que condicionarán al plan diseñado por la Inteligencia Artificial que construirá sobre los alimentos guardados y modelados sobre la base datos; ya que cada uno de ellos tiene etiquetas que los sitúan en ciertos grupos de alérgenos, familia de alimentos, y también especifican los macronutrientes y propiedades que contienen.

Palabras clave

FIXEAR: Plataforma web, nutrición, gestión de clientes, base de datos relacional, alertas inteligentes, gamificación, adherencia, formularios personalizables, lista de la compra, reconocimiento de alimentos, marketplace, automatizaciones, centralización

Abstract

Development of an Intelligent Web Platform for Nutritionists

This Bachelor's Thesis presents the design of NutriApp, a web platform aimed at centralizing the workflow of professional nutritionists. The idea emerged from observing first-hand how a nutritionist manages his daily practice: his information was scattered across physical notebooks, Google Forms, Google Calendar, his clinic's own platform, phone notes, and WhatsApp. Getting the full picture of a single client meant manually cross-referencing data from all of these tools, which was inefficient and made it easy for important details to slip through the cracks.

Based on this real problem, the goal was to design a web application that brings all of these functionalities together into a single point of access. The platform is built around two separate panels: one for the nutritionist, where they can manage clients, appointments, forms, nutritional plans, and private notes from one place; and one for the client, providing a dedicated interface to view their plan, log daily habits, communicate with their nutritionist, and track their progress.

Beyond replacing the scattered tools, NutriApp introduces features that are only possible when all data is connected: a smart alert system that detects risk patterns from client data, gamified daily check-ins with streaks to encourage plan adherence, shopping lists with substitutions based on nutritional equivalence, a customizable form builder with associated alert rules, an integrated food database with barcode scanning and AI-based photo recognition, a marketplace of educational resources that gives the nutritionist an additional monetization channel, and an automations engine that allows them to model recurring procedures of their clinical practice. Communication with the client happens through an integrated chat with video and voice calls, removing the dependency on external tools. The database design, developed in collaboration with the thesis supervisors, reflects the complexity of these features and constitutes the core of the project.

Keywords

Web platform, nutrition, client management, relational database, smart alerts, gamification, adherence, customizable forms, shopping list, food recognition, marketplace, automations, centralization

Índice

1. Introducción	1
1.1. Motivación	1
1.2. Objetivos	2
1.3. Plan de trabajo	3
2. Estado de la Cuestión	7
2.1. Criterios de comparación	7
2.2. Análisis de competidores	8
2.2.1. Plataformas integrales	8
2.2.2. Plataforma centrada en el seguimiento de alimentos	9
2.3. Resumen comparativo	9
2.4. Posicionamiento de NutriApp	10
2.5. Alcance de las integraciones externas	11
3. Análisis de Requisitos	13
3.1. Reformulación del alcance	13
3.1.1. Criterio de decisión aplicado en el triaje	14
3.1.2. Bloques eliminados completos y su justificación	15
3.2. Requisitos no funcionales	17
3.2.1. Requisitos no funcionales generales	17
3.2.2. Requisitos no funcionales específicos del copiloto IA	18
3.3. Requisitos funcionales	20
3.3.1. Gestión de clientes	20
3.3.2. Calendario y citas online	20
3.3.3. Mensajería	21
3.3.4. Planes nutricionales: estructura base	21
3.3.5. Copiloto de planes nutricionales (<i>innovación clave</i>)	22
3.3.6. Check-in mínimo de adherencia	23
3.3.7. Progreso del cliente	23
3.3.8. Base de datos de alimentos	24
3.3.9. Dashboard de métricas del copiloto	24
3.4. Requisitos pospuestos a trabajo futuro	25

3.5. Dependencias funcionales	26
3.6. Funcionalidades resultantes	27
4. Diseño del sistema	29
4.1. Arquitectura general	29
4.2. Capa de datos	31
4.3. Capa de seguridad	33
4.4. Capa de servicio	34
4.5. Despliegue	35
4.6. Validación y testing	36
5. El motor de generación	37
5.1. La programación con restricciones	37
5.2. Formulación del problema	38
5.2.1. El problema y su contrato	38
5.2.2. Elección de tecnología	38
5.2.3. Conjuntos y parámetros	39
5.2.4. Variables de decisión	40
5.2.5. Trabajo en enteros	40
5.3. Restricciones del modelo	41
5.3.1. Restricciones estructurales	41
5.3.2. La capa de plausibilidad del catálogo	43
5.3.3. Restricciones clínicas y restricciones de catálogo	44
5.4. El catálogo de restricciones clínicas	45
5.5. Función objetivo y resolución	46
5.6. Diagnóstico de infactibilidad	47
5.7. El validador determinista	48
6. El copiloto de IA	49
6.1. El flujo del copiloto	49
6.2. Del texto a las restricciones	50
6.3. La elección del modelo de lenguaje	51
6.4. Revisión y arbitraje	52
6.5. De la petición al plan firmado	52
7. Evaluación	55
8. Conclusiones y Trabajo Futuro	57
Introduction	59
Conclusions and Future Work	61
Bibliografía	63

A. Catálogo original de requisitos (versión V1)	65
A.1. Requisitos no funcionales (V1)	65
A.2. Requisitos funcionales (V1)	67
A.2.1. Gestión de clientes	67
A.2.2. Calendario y citas	67
A.2.3. Mensajería	68
A.2.4. Formularios personalizables	68
A.2.5. Sistema de alertas inteligentes (<i>innovación clave V1</i>)	68
A.2.6. Planes nutricionales	69
A.2.7. Lista de la compra inteligente (<i>innovación clave V1</i>)	69
A.2.8. Check-in diario y gamificación (<i>innovación clave V1</i>)	70
A.2.9. Seguimiento y progreso	70
A.2.10. Informes y análisis	71
A.2.11. Base de datos de alimentos con reconocimiento inteligente (<i>innovación clave V1</i>)	71
A.2.12. Marketplace de recursos educativos (<i>innovación clave V1</i>)	71
A.2.13. Automatización de procesos (<i>innovación clave V1</i>)	72
A.3. Dependencias funcionales (V1)	73
A.4. Funcionalidades resultantes (V1)	74
A.5. Nota final sobre el alcance definitivo	76
B. Título del Apéndice B	77
C. Formalización completa del motor de generación	79
C.1. Umbrales y pesos del modelo	79
C.2. Variables y enlace	80
C.3. Restricciones estructurales (R1 a R8)	81
C.4. Restricciones de plausibilidad (R9 a R15)	82
C.5. Los 16 tipos configurables	83
C.6. Función objetivo	85
C.7. Mapa de restricciones a código	86

Índice de figuras

4.1. Arquitectura del sistema: la capa de presentación, la de datos y la de servicio, con el worker de generación como pieza que corre en un ordenador local.	30
4.2. La tabla de restricciones como interfaz: tres productores escriben filas del mismo vocabulario y un único consumidor las aplica sin distinguir su origen.	31
4.3. Modelo de datos: las tablas del núcleo con sus relaciones y el resto del esquema agrupado por dominios. Las restricciones apuntan además a su objetivo (un alimento, una etiqueta o un nutriente).	32
4.4. Dos roles sobre las mismas tablas: cada uno entra con su sesión y ve o escribe lo que sus políticas permiten, con tres tablas como ejemplo.	34
4.5. Despliegue: tres patas en planes gratuitos de nube y el worker en una máquina local, todas comunicadas a través de la base de datos o de llamadas de servidor a servidor.	35
6.1. El flujo del copiloto. La fila superior son las decisiones del profesional y la inferior el trabajo del sistema: ningún resultado avanza hacia el cliente sin pasar antes por una decisión suya.	50

Índice de tablas

2.1. Cobertura funcional de los principales competidores frente a NutriApp.	10
3.1. Reformulación del alcance del TFG: del catálogo original al definitivo.	14
3.2. Bloques completos eliminados del alcance tras la reformulación.	16
3.3. Requisitos funcionales — Gestión de clientes.	20
3.4. Requisitos funcionales — Calendario y citas online.	21
3.5. Requisitos funcionales — Mensajería.	21
3.6. Requisitos funcionales — Planes nutricionales: estructura base.	22
3.7. Requisitos funcionales — Copiloto de planes nutricionales.	23
3.8. Requisitos funcionales — Check-in mínimo de adherencia.	23
3.9. Requisitos funcionales — Progreso del cliente.	24
3.10. Requisitos funcionales — Base de datos de alimentos.	24
3.11. Requisitos funcionales — Dashboard de métricas del copiloto.	24
3.12. Requisitos individuales pospuestos a trabajo futuro.	26
3.13. Dependencias funcionales de NutriApp V2. La funcionalidad marcada con * es la única innovación clave del TFG.	27
5.1. Efecto de las reglas de sentido común sobre un mismo caso (7 días, 5 comidas).	42
5.2. Catálogo de restricciones clínicas: los 16 tipos.	45
A.1. Requisitos funcionales V1 — Gestión de clientes.	67
A.2. Requisitos funcionales V1 — Calendario y citas.	67
A.3. Requisitos funcionales V1 — Mensajería.	68
A.4. Requisitos funcionales V1 — Formularios personalizables.	68
A.5. Requisitos funcionales V1 — Sistema de alertas inteligentes.	69
A.6. Requisitos funcionales V1 — Planes nutricionales.	69
A.7. Requisitos funcionales V1 — Lista de la compra inteligente.	70
A.8. Requisitos funcionales V1 — Check-in diario y gamificación.	70
A.9. Requisitos funcionales V1 — Seguimiento y progreso.	70
A.10. Requisitos funcionales V1 — Informes y análisis.	71
A.11. Requisitos funcionales V1 — Base de datos de alimentos.	71
A.12. Requisitos funcionales V1 — Marketplace de recursos educativos.	72

A.13.Requisitos funcionales V1 — Automatización de procesos.	72
A.14.Dependencias funcionales V1: funcionalidades y sus requisitos. Las funcionalidades marcadas con * son innovaciones clave del plantea- miento original.	73
C.1. Umbrales del modelo y parámetros de resolución.	80
C.2. Mapa a código: modelo y reglas invariantes.	87
C.3. Mapa a código: catálogo clínico, objetivo y resolución.	87

Introducción

Este capítulo presenta el punto de partida del proyecto: la motivación que lo origina, los objetivos que persigue y el plan de trabajo que ha guiado su desarrollo.

1.1. Motivación

Al plantear este Trabajo de Fin de Grado, existía una premisa clara: que el resultado tuviera una aplicación real. Resolver un problema inventado para cumplir un requisito académico no me motivaba, así que busqué la forma de utilizar los conocimientos adquiridos durante la carrera y orientarlos hacia una necesidad concreta de alguien cercano. La idea era que el trabajo sirviera como ejercicio técnico y, al mismo tiempo, como solución a un problema que alguien experimentaba en su día a día profesional.

Ese alguien resultó ser un amigo que trabaja como nutricionista. En una conversación sobre su rutina diaria, encontré un problema que él mismo no percibía como tal, porque lo había normalizado: su flujo de trabajo estaba fragmentado entre cinco o más herramientas que no se comunicaban entre sí. Utilizaba formularios de Google para recoger información de sus clientes, Google Calendar para gestionar las citas, WhatsApp como canal de comunicación directa, un cuaderno físico para sus notas privadas sobre cada caso, y la plataforma de gestión de su clínica para el resto. Para hacerse una idea completa de la situación de un solo cliente, tenía que consultar todas estas fuentes manualmente y reconstruir la información por su cuenta. Además, gran parte de su tiempo se iba en responder consultas repetitivas como: si se podía sustituir un ingrediente, cuándo debería ser la próxima cita, y miles de dudas varias sobre cómo arreglar el plan tras haberse saltado/cambiado comidas; cuestiones casi triviales que podrían resolverse sin su intervención directa.

A partir de esa conversación surgió la idea central de este proyecto: diseñar una plataforma que centralizara todas esas herramientas dispersas en un único punto. Si toda la información de los clientes (formularios, citas, mensajes, notas, planes nutricionales, registros diarios) viviera en una misma base de datos, se eliminaría la división de fuentes y se abrirían nuevas posibilidades como detectar patrones de forma automática, generar alertas cuando un cliente muestra señales de abandono, o construir un perfil completo sin tener que cruzar datos a mano. En definitiva,

que el nutricionista pudiera dedicar su tiempo a lo que realmente importa: diseñar dietas, acompañar a sus clientes y lograr que los tratamientos funcionen en lugar de invertirlo en tareas administrativas que una aplicación bien diseñada puede resolver.

Esa visión inicial, una plataforma que centralizara todo el flujo de trabajo, guió la primera versión del proyecto: se catalogó todo lo que una herramienta así podría ofrecer y se diseñó un prototipo completo con ello, desde formularios personalizables hasta un marketplace de recursos educativos. Al presentar la propuesta a más nutricionistas en activo, sus inquietudes ayudaron a repensar el enfoque. Lo que de verdad demandaban no era tenerlo todo en un mismo sitio, que obviamente también sería de gran ayuda, sino que también encontraban un cuello de botella con la tarea que más tiempo les consume como nutricionistas: elaborar cada dieta a medida. La ficha del cliente (sus datos, mediciones y restricciones) les importaba como apoyo de esa tarea; la comunicación con el cliente la veían como un complemento, no como el núcleo; y la centralización del resto de sus procesos, en la práctica, depende de la clínica en la que trabajan, no de la herramienta que ellos elijan.

Ese contraste con la demanda real me llevó a reformular el proyecto que se detalla en el capítulo 3. El centro dejó de ser la plataforma total y pasó a ser un copiloto de generación de planes nutricionales a medida: el nutricionista describe lo que necesita en su propio idioma, el sistema construye un borrador que cumple restricciones verificables, y el profesional lo revisa, lo ajusta y lo firma antes de que llegue al cliente. La plataforma sigue existiendo alrededor (clientes, citas, mensajes, seguimiento) pero como soporte de ese núcleo, no como fin en sí misma.

1.2. Objetivos

Este Trabajo de Fin de Grado se desarrolla en el marco del Doble Grado en Ingeniería Informática y Administración y Dirección de Empresas, lo cual permite abordarlo desde dos perspectivas complementarias: la técnica, que abarca desde el diseño de la base de datos hasta la integración de un motor de optimización y modelos de lenguaje; y la de negocio, aplicando herramientas de análisis de mercado y viabilidad de producto propias de la formación en ADE. El objetivo no es simplemente cumplir un requisito académico, sino aprovechar esta doble formación para construir un proyecto totalmente funcional con aplicación real.

El objetivo general de este trabajo es diseñar, construir y evaluar una plataforma web para nutricionistas cuyo núcleo es un copiloto de generación de planes nutricionales: el profesional expresa en su propio idioma las necesidades de un cliente, la inteligencia artificial las traduce a restricciones formales, un motor de optimización construye un borrador que las satisface y un validador independiente lo audita, quedando siempre la decisión final en manos del profesional. Se busca recorrer el proceso completo de un producto real, refinando la idea con profesionales y con el objetivo de despliegue público y la evaluación empírica, para que el resultado sea técnicamente sólido y también defendible como herramienta real.

Para alcanzar este objetivo general, se plantean los siguientes objetivos específicos:

1. Analizar el flujo de trabajo real de un nutricionista profesional para extraer

los requisitos funcionales del sistema. Esto implica comprender cómo trabaja en su día a día, qué herramientas utiliza actualmente, qué problemas enfrenta y qué necesita realmente. De esta forma, los requisitos no se inventan, sino que se extraen de la observación directa de un caso real, para que el diseño responda a necesidades concretas y no a suposiciones.

2. Realizar un estudio de mercado y análisis de la competencia en el ámbito de las plataformas de nutrición, aplicando conocimientos propios de la formación en ADE. Esto incluye identificar los principales competidores, analizar sus fortalezas y debilidades, detectar carencias y oportunidades en el mercado, y posicionar NutriApp con funcionalidades innovadoras y diferenciadoras que aporten valor real.
3. Diseñar la arquitectura del sistema y su modelo de datos: dos paneles (nutricionista y cliente) sobre una misma base de datos, con el aislamiento entre profesionales garantizado en la propia capa de datos, y con una tabla de restricciones que actúa como interfaz entre quien las produce (el profesional o la IA) y el motor que las consume.
4. Construir un motor de generación de planes basado en programación con restricciones, acompañado de un validador clínico determinista e independiente, capaz de funcionar por sí solo, sin depender de ningún componente de inteligencia artificial.
5. Integrar la capa de IA que hace usable ese motor en lenguaje natural: traducción de las peticiones del profesional a restricciones formales, validación previa de los mensajes y generación asíncrona manteniendo al profesional como árbitro de cada decisión.

1.3. Plan de trabajo

El desarrollo de este proyecto ha seguido un proceso iterativo en el que cada fase se apoya en los resultados de la anterior. A continuación se describen las fases, desde la investigación inicial con profesionales hasta la evaluación del sistema final.

Fase 1 — Investigación con nutricionistas reales

El punto de partida fue entender en profundidad el problema que se quería resolver. Mantuve una serie de conversaciones con el nutricionista cuyo caso motivó el proyecto para conocer a fondo su método de trabajo: qué herramientas utilizaba, en qué puntos perdía más tiempo y qué información se le escapaba por tenerla repartida entre plataformas desconectadas.

Tras consultar con los tutores, me recomendaron contactar a más nutricionistas para no basar el producto en la experiencia de una sola persona. Por ello contacté a través de LinkedIn con otros profesionales del sector y les pregunté lo mismo. Esta ronda de consultas me permitió contrastar ideas para construir una visión del producto más amplia y sólida.

Fase 2 — Análisis de competencia y mercado

Una vez definidas las necesidades desde el lado del usuario, era el momento de investigar el mercado. Obviamente, ya existían soluciones muy similares, así que convenía saber quiénes eran los competidores, qué ofrecían y dónde dejaban huecos sin cubrir. Analicé las principales plataformas del sector (Nutrium, Indya, Dietopro, Nutrimeal y Practice Better) estudiando sus funcionalidades, modelos de negocio, puntos fuertes y carencias, para detectar qué aspectos se habían quedado fuera del radar en la fase anterior y si las necesidades que yo había recogido estaban ya cubiertas o existía una oportunidad real. Al final de esta fase, la idea del producto estaba mucho más definida, con una propuesta de valor más clara.

Fase 3 — Diseño visual del prototipo

Con los requisitos extraídos, los traduje en una interfaz concreta: los mockups de la aplicación. El objetivo de las pantallas era doble; que el nutricionista pudiera gestionar todo su trabajo desde un único punto de acceso (clientes, citas, formularios, planes, mensajes y notas) y que el cliente contara con herramientas para consultar su plan, registrar sus hábitos y seguir su progreso. Cada pantalla cubría casos de uso identificados en las fases anteriores.

El conjunto de pantallas seguía el mismo sistema de diseño para mantener la coherencia visual. Se revisó lo ya diseñado para incorporar mejoras detectadas durante el análisis de requisitos, como las vistas diaria, semanal y mensual del calendario o la pantalla de ajustes del cliente. El resultado fue un primer prototipo navegable con los dos paneles interconectados y todos los requisitos funcionales iniciales presentes.

Fase 4 — Segunda iteración: competencia en profundidad y nuevas funcionalidades

Tras completar el primer prototipo, analicé por segunda vez la competencia, esta vez contrastando NutriApp con siete plataformas de referencia. De ese análisis surgieron cuatro incorporaciones al diseño: un motor de automatizaciones configurable, un marketplace de recursos educativos, una base de datos de alimentos con reconocimiento inteligente y las videollamadas integradas en el chat; además de refinamientos varios, como la ficha detallada del cliente. El resultado fue un prototipo navegable completo de veinticuatro pantallas, con los sesenta y un requisitos funcionales representados y la navegación cableada en Figma, de forma que un evaluador externo podía recorrerlo como si fuera una aplicación real.

Fase 5 — Validación con usuarios reales

Con el prototipo completo, preparé dos guías de prueba de usabilidad (nutricionista y cliente), cada una con doce flujos de navegación formulados como pequeños retos (“quieres consultar la adherencia media de tus clientes, ¿a qué sección del menú lateral irías?”). El evaluador debía simular el flujo de navegación en cada caso y marcar si le resultó obvia, confusa o si directamente no sabía cómo llegar.

Fase 6 — Reformulación del alcance

Inesperadamente, el prototipo completo "final" (para el plan que había en ese momento) resultó ser la mejor herramienta para contrastar la idea de implementación. Al presentarlo a más nutricionistas y hablar más con ellos para recibir feedback, quedó claro que el valor no estaba en abarcar todas las funcionalidades posibles, sino en automatizar la elaboración de dietas a medida, la tarea que más tiempo les consume.

Además, los tutores cuando probaron el prototipo, me hicieron ver la complejidad y cantidad de funcionalidades que quería programar yo solo. Esto me hizo bajar los pies a tierra y así darme cuenta de que debía enfocarme en cubrir la necesidad primaria de los nutricionistas y centrar mis esfuerzos en esa dirección. Así que, decidí acotar el alcance drásticamente a lo que sostiene esa tarea y concentrar el esfuerzo técnico en el copiloto de generación de planes, que es donde se encontraba el valor real. El capítulo 3 documenta el triaje completo y sus criterios.

Fase 7 — Diseño e implementación de la base de datos

Siguiendo el orden recomendado por Alejandro y Pablo: primero la base de datos, después su conexión, por último las pantallas; se diseñó e implementó el modelo de datos completo sobre PostgreSQL gestionado: las tablas de catálogos, personas, restricciones y planes; las políticas de seguridad a nivel de fila que garantizan el aislamiento entre profesionales; y los datos de arranque necesarios para trabajar con casos realistas desde el primer día.

Fase 8 — Construcción de la aplicación web

Sobre esa base construí la aplicación en dos pasos. Primero, la conexión de las dos piezas que hablan con la base de datos: el backend en Python y el frontend en Next.js, cada uno verificado con lecturas y escrituras reales antes de dibujar una sola pantalla. Después, el panel del nutricionista sección a sección (clientes, planes, alimentos, calendario, mensajería y ajustes), siempre sobre datos reales y sin simular funcionalidad que no existiera: si una pantalla mostraba algo, había una tabla detrás que lo sostenía.

La aplicación se desplegó públicamente en cuanto tuvo sus primeras secciones, siguiendo la recomendación de los tutores, lo que permitió enseñarla y probarla en condiciones reales lo antes posible. Al cerrar esta fase comparé el prototipo de Figma con lo implementado y revisé las diferencias, para no olvidarme de elementos y revisar de nuevo si algunos de ellos no eran necesarios, justificando la divergencia.

Fase 9 — Motor de generación de planes

El corazón técnico del proyecto: la formalización de la dieta como un problema de satisfacción de restricciones. Tras consultar con Alejandro cuál sería el mejor enfoque, me explicó que su implementación debería consistir en un solver de programación con restricciones, un validador determinista independiente que audita cada

plan antes de darlo por bueno, y me indicó que podría añadir también un diagnóstico de infactibilidad cuando las peticiones son incompatibles entre sí. El motor funciona sin ningún componente de inteligencia artificial, lo que permitió verificarlo dentro de la propia web (creando un apartado para modificar los parámetros que se usan para generar el plan con el motor).

Fase 10 — Capa de inteligencia artificial

Con el motor verificado se añadió la capa que lo hace usable en lenguaje natural: el traductor que convierte las peticiones del nutricionista en restricciones formales, las validaciones previas que examinan cada mensaje antes de traducirlo, la generación asíncrona y el estilo clínico que cada profesional define una vez y se aplica a todos sus planes. Todo esto se implementó mediante consultas a la inteligencia artificial de Ollama.

Fase 11 — Panel del cliente

Después desarrollé el segundo rol de la aplicación: el panel del cliente. El cliente accede a su plan una vez firmado, marca las comidas que ha completado, registra su peso y solicita citas; toda esa información vuelve al panel del nutricionista como métricas de seguimiento. Con esta fase queda cerrado el ciclo completo entre profesional y cliente y toda la información que debía ser compartida entre ambos roles ya estaba implementada.

Fase 12 — Evaluación

La evaluación acompañó a todo el desarrollo y solo le queda su última pieza. Hasta ahora incluye un estudio pre-registrado sobre el efecto del formato de los mensajes en la traducción (con un resultado negativo que evitó construir un módulo innecesario), mediciones de rendimiento del motor en el entorno de producción al crecer el catálogo de alimentos, y una auditoría sistemática de la calidad nutricional de los planes generados, cuyo antes y después justificó una capa entera de reglas nuevas del motor. Queda pendiente, con el entorno y las guías ya preparados, una prueba guiada con nutricionistas reales sobre la aplicación desplegada; sus conclusiones cerrarán el ciclo que empezó en la Fase 1: del profesional al prototipo, del prototipo al producto, y del producto de vuelta al profesional.

Estado de la Cuestión

NutriApp no nace en un mercado vacío. Existen varias plataformas comerciales que ya intentan resolver, total o parcialmente, el problema de centralizar el flujo de trabajo de un nutricionista profesional. Antes de diseñar una propuesta propia, era razonable estudiar qué se ofrece ya, qué funciona bien, qué se ha quedado a medio camino y, sobre todo, dónde están los huecos que justifican construir algo nuevo en lugar de adoptar un producto existente.

Este capítulo recoge ese análisis. Se han estudiado las cuatro plataformas más representativas del sector —las tres principales soluciones integrales del mercado profesional y la referencia en seguimiento de alimentos— evaluándolas con un conjunto homogéneo de criterios funcionales. No se pretende elaborar un *benchmark* exhaustivo, sino tener una referencia clara que permita explicar las decisiones de diseño que se toman más adelante en los capítulos 3 y 4, y dejar fundamentado por qué NutriApp incorpora ciertas funcionalidades y descarta otras de forma deliberada.

2.1. Criterios de comparación

Para que la comparación tenga sentido, todos los competidores se han revisado bajo el mismo conjunto de criterios funcionales:

- Gestión de clientes (perfiles, historial, notas privadas).
- Calendario y citas con soporte multi-clínica.
- Mensajería integrada con el cliente.
- Constructor de formularios personalizables.
- Sistema de alertas configurables por el profesional.
- Constructor de planes nutricionales y biblioteca de recetas.
- Lista de la compra generada desde el plan.
- Base de datos de alimentos con buscador.

- Reconocimiento automático de alimentos (código de barras, fotografía).
- Marketplace de recursos educativos.
- Motor de automatizaciones (reglas con disparadores).
- Idioma del producto y mercado objetivo.
- Modelo de precios e integraciones externas.

A estos criterios se añade una valoración cualitativa de la interfaz, la curva de aprendizaje y el público al que va dirigida cada plataforma, ya que son aspectos relevantes para entender las decisiones de diseño de cada producto aunque no se reflejen en una tabla de funcionalidades.

2.2. Análisis de competidores

2.2.1. Plataformas integrales

Nutrium. Nutrium (Nutrium, 2025) es la plataforma de referencia del sector. Nace en Portugal y opera en más de cien países, con una base estimada de 120.000 profesionales. Su gran fortaleza es la solidez como herramienta integral: cuenta con una aplicación de paciente bien construida, mensajería interna, evaluaciones nutricionales completas, biblioteca de recetas, plantillas de planes y cumplimiento normativo de privacidad sanitaria. Su interfaz es limpia y profesional, lo que la convierte en la referencia visual del sector. Sus debilidades son notables para el perfil de usuario al que se dirige NutriApp: el plan completo es caro para un nutricionista autónomo, la curva de aprendizaje es alta porque incorpora muchas funcionalidades poco utilizadas en consultas pequeñas, y su sistema de notificaciones al profesional es genérico —no permite definir reglas personalizadas sobre los formularios—. NutriApp toma de Nutrium su **lenguaje visual y la organización general del panel del nutricionista**, pero descarta su modelo de funcionalidades exhaustivas a favor de un conjunto más reducido y mejor conectado entre sí.

Practice Better. Practice Better (Practice Better, 2025) es una plataforma canadiense con fuerte presencia en Estados Unidos. Está pensada como sistema integral para profesionales de la salud y la nutrición, con cumplimiento HIPAA, integraciones con Zoom, Google Calendar y Stripe, formularios personalizables y un módulo de gestión de pagos. Su mayor punto fuerte es el grado de personalización: el profesional puede construir plantillas reutilizables para casi cualquier flujo de trabajo, y dispone de un sistema básico de automatizaciones que permite encadenar acciones tras determinados eventos. Su mayor debilidad es que está pensado para el mercado norteamericano —no existe versión en castellano y su modelo de facturación está orientado a clínicas privadas estadounidenses—, y que sus formularios, aunque flexibles, no están conectados a ningún tipo de motor de alertas: las respuestas se almacenan, pero el profesional debe revisarlas manualmente. NutriApp toma de Practice Better la idea de un **motor de automatizaciones configurable por el**

profesional, lo amplía vinculándolo al sistema de alertas y lo trae a un mercado donde no existe equivalente en castellano.

Healthie. Healthie (Healthie, 2025) es otra plataforma estadounidense, también orientada a clínicas y consultas privadas, que destaca por su *marketplace de recursos*: los profesionales pueden publicar guías, plantillas, programas y materiales formativos dentro de la propia plataforma, ofreciendo a sus clientes contenido educativo curado y, al mismo tiempo, una vía de monetización adicional para el nutricionista. Su interfaz es moderna y la integración del marketplace con el resto del flujo de trabajo es uno de sus puntos diferenciadores. Sin embargo, también tiene los problemas de Practice Better: producto exclusivamente en inglés, modelo de precios elevado y enfoque en el mercado norteamericano, además de una experiencia de cliente que se queda corta en la parte de seguimiento diario y gamificación de la adherencia. NutriApp toma de Healthie la idea del **marketplace integrado dentro del flujo**, con la particularidad de que el cliente ve una sección destacada con los recursos publicados específicamente por su propio nutricionista, no solo el catálogo general.

2.2.2. Plataforma centrada en el seguimiento de alimentos

Cronometer Pro. Cronometer (Cronometer Software Inc., 2025) es originalmente una aplicación de *food tracking* orientada al consumidor, con una versión profesional (Cronometer Pro) que permite al nutricionista vincular cuentas de clientes y revisar su registro alimentario. Su mayor fortaleza es la **base de datos de alimentos**, una de las más completas del sector, con información de micronutrientes que muchas plataformas no incluyen. También dispone de buscador por código de barras, sincronizado con bases públicas. Su debilidad principal es que es esencialmente una herramienta de tracking: no construye planes nutricionales completos, no tiene formularios personalizables, no gestiona citas y no incluye mensajería real con el profesional. Es una pieza valiosa, pero no una plataforma integral. NutriApp toma de Cronometer la idea de **integrar una base de datos de alimentos accesible y un buscador por código de barras**, y la combina con un servicio externo de reconocimiento de alimentos por fotografía mediante inteligencia artificial —una funcionalidad presente ya en algunos productos emergentes del sector, pero todavía poco extendida en plataformas integrales como las anteriores—.

2.3. Resumen comparativo

La tabla 2.1 sintetiza qué cubre cada una de las cuatro plataformas frente a NutriApp en los criterios funcionales más relevantes. Se utiliza “Sí” cuando la funcionalidad está presente y bien integrada, “Parc.” (parcial) cuando existe en una versión limitada o aislada, y “No” cuando la funcionalidad no está disponible.

Funcionalidad	NutriApp	Nutrium	P. Better	Healthie	Cron. Pro
Gestión de clientes	Sí	Sí	Sí	Sí	Parc.
Calendario y citas	Sí	Sí	Sí	Sí	No
Mensajería integrada	Sí	Sí	Sí	Sí	No
Videollamada y voz en chat	Sí	No	Parc.	Parc.	No
Formularios personalizables	Sí	Sí	Sí	Sí	No
Alertas configurables por regla	Sí	No	No	No	No
Planes nutricionales	Sí	Sí	Sí	Sí	No
Lista de compra inteligente	Sí	Parc.	No	No	No
Base de alimentos con buscador	Sí	Sí	No	No	Sí
Escaneo de código de barras	Sí	No	No	No	Sí
Reconocimiento por fotografía	Sí	No	No	No	No
Check-in diario gamificado	Sí	No	No	No	No
Marketplace de recursos	Sí	No	No	Sí	No
Motor de automatizaciones	Sí	No	Parc.	Parc.	No
Comunidad entre clientes	Sí	No	No	No	No
Disponible en castellano	Sí	Sí	No	No	No

Tabla 2.1: Cobertura funcional de los principales competidores frente a NutriApp.

La tabla deja a la vista varios patrones útiles para justificar el diseño. Las tres plataformas integrales cubren con solvencia las funcionalidades de gestión profesional, pero ninguna ofrece alertas configurables por regla, lista de la compra inteligente, check-in gamificado, reconocimiento de alimentos por fotografía ni canal de comunidad entre clientes. Cronometer Pro es fuerte precisamente en el bloque que las otras ignoran —base de datos de alimentos y escáner de código de barras— pero no es una herramienta integral para gestionar una consulta. Ninguna de las cuatro cubre simultáneamente todas las dimensiones que NutriApp aborda, y la única que está disponible en castellano es Nutrium.

2.4. Posicionamiento de NutriApp

A partir de este análisis, NutriApp se posiciona como una plataforma integral pensada para el mercado hispanohablante, que toma prestados elementos de las mejores referencias del sector y los combina con un conjunto de innovaciones que ningún competidor cubre por sí solo. La conexión entre cada una de las ocho funcionalidades clave del proyecto y el hueco del mercado al que responde es directa:

- **El lenguaje visual y la organización general del panel** se inspiran en Nutrium, la referencia estética del sector. NutriApp adopta su limpieza visual sin replicar su densidad funcional.
- **El sistema de alertas inteligentes con reglas configurables por formulario** no existe en ninguna plataforma analizada. Este es el principal diferenciador del proyecto y emerge directamente de la observación del flujo de trabajo real del nutricionista.

- El **check-in diario gamificado con rachas de adherencia** tampoco tiene equivalente en la competencia. Ninguna de las cuatro plataformas estudiadas incorpora una mecánica de gamificación real.
- La **lista de la compra inteligente con sustituciones por equivalencia nutricional** es otra propuesta única. Nutrium genera listas básicas, pero ninguna plataforma sugiere alternativas inteligentes cuando un ingrediente no está disponible.
- El **motor de automatizaciones configurable** se inspira en Practice Better, pero NutriApp lo amplía conectándolo al sistema de alertas: la misma estructura de reglas que dispara una alerta puede disparar también una acción automática.
- El **marketplace de recursos educativos** se inspira en Healthie. NutriApp lo integra dentro del flujo del cliente con una sección destacada que muestra los recursos publicados específicamente por su propio nutricionista, no solo el catálogo general.
- La **base de datos de alimentos con escáner de código de barras** se inspira en Cronometer Pro, y se complementa con un servicio de **reconocimiento de alimentos por fotografía** mediante inteligencia artificial. La combinación de ambas vías de captura no aparece en ninguna de las plataformas integrales analizadas.
- El **canal de comunidad entre clientes del mismo nutricionista** es una propuesta original de NutriApp que ningún competidor analizado contempla. Funciona como apoyo entre pares, con aislamiento garantizado entre consultas de profesionales distintos.

El conjunto resultante no es una suma de funcionalidades, sino una plataforma donde cada pieza se conecta con las demás a través de una base de datos común. Esa conexión es precisamente lo que permite que el sistema de alertas se alimente de los formularios y los check-ins, que la lista de la compra se genere automáticamente del plan, y que las automatizaciones puedan reaccionar a cualquier evento del sistema sin intervención manual del profesional.

2.5. Alcance de las integraciones externas

Para cerrar este capítulo conviene aclarar el alcance de las integraciones externas que el diseño contempla. Muchas plataformas profesionales del sector presumen de integraciones con ecosistemas como Apple Health, Google Fit, Fitbit o Garmin, además de servicios de videollamada de terceros y pasarelas de pago. NutriApp no descarta esas integraciones a futuro, pero las deja fuera del alcance de esta iteración del TFG: tienen sentido cuando el producto está implementado y empieza a tener tracción real, no en la fase de diseño, donde añadirían complejidad sin contribuir al eje académico del trabajo (el diseño de la base de datos en el marco de la asignatura de Ampliación de Bases de Datos).

Sí entran en el diseño dos servicios externos concretos, ambos consumidos a través de APIs REST, porque su salida enriquece directamente el modelo de datos del proyecto:

- **Open Food Facts** (Open Food Facts, 2025). Base de datos abierta y colaborativa de productos alimentarios. Su API pública y gratuita permite consultar productos por código de barras o por nombre y devuelve la información nutricional en formato JSON. Es la fuente que alimenta el buscador de la base de datos de alimentos y el escáner de código de barras.
- **Servicio de reconocimiento de alimentos por imagen.** El reconocimiento automático de alimentos a partir de una fotografía se resuelve con una API REST especializada (existen varios proveedores comerciales con interfaces equivalentes, como LogMeal, Passio o Foodvisor), que recibe la imagen y devuelve los alimentos identificados con sus macronutrientes estimados.

El resto de integraciones —wearables y apps de salud, pasarelas de pago para el marketplace, servicios de videollamada de terceros para los botones del chat— quedan documentadas como trabajo futuro y se acometerían en la fase de implementación, cuando la plataforma esté construida y empiece a probarse con usuarios reales.

Capítulo 3

Análisis de Requisitos

A partir de la investigación con nutricionistas reales y del análisis de mercado descritos en el plan de trabajo, se extrajeron los requisitos que debe cumplir la plataforma. Este capítulo recoge tanto las restricciones de calidad del sistema (requisitos no funcionales) como las funcionalidades concretas que debe ofrecer (requisitos funcionales), sus dependencias, las funcionalidades resultantes y las decisiones tecnológicas que lo sostienen.

3.1. Reformulación del alcance

El catálogo inicial de requisitos recogía **61 requisitos funcionales y 7 no funcionales**, estructurados alrededor de ocho innovaciones clave, y respondía a una pregunta demasiado ambiciosa: todo lo que una plataforma de gestión para nutricionistas podría llegar a ofrecer. Este catálogo se conserva en el Anexo A como registro del punto de partida del proyecto.

Como ya comenté, las necesidades más importantes de los profesionales fueron lo que obligaron a matizarlo. Tras una reunión con Pablo y Alejandro en abril de 2026, acordamos una reducción a unos 44 requisitos de primer nivel, ya que era inviable abarcarlo todo en un desarrollo en solitario.

Al descubrir la demanda real de los nutricionistas, automatizar la elaboración de cada dieta a medida (la tarea que más tiempo les quita); clasifiqué la importancia de las funcionalidades relacionadas: la ficha del cliente como apoyo de esa tarea es útil; la comunicación con el cliente es un complemento, no el núcleo; y la centralización del resto de sus procesos depende de las clínicas en las que trabajan, no de la herramienta.

Por ello, la decisión resultante no fue un recorte por falta de tiempo, sino más bien una elección informada por ese feedback: **concentrar todo el esfuerzo técnico en una única funcionalidad diferencial de alta complejidad**, un *copiloto* de generación de planes nutricionales en el que la inteligencia artificial propone, y el profesional revisa, ajusta y firma cada plan antes de entregarlo.

Este giro obliga a reformular los requisitos en dos direcciones complementarias:

- **Recorte en amplitud.** Se elimina por completo la implementación de bloques

que no contribuyen al núcleo del copiloto o a su evaluación: formularios personalizables, sistema de alertas inteligentes, lista de la compra, marketplace de recursos educativos, motor de automatizaciones configurable, reconocimiento de alimentos por imagen, videollamadas y chat entre clientes. Varios bloques adicionales se conservan en forma reducida (calendario, check-in, progreso, informes).

- **Ampliación en profundidad.** Se añaden **ocho nuevos requisitos funcionales** que describen el copiloto con el detalle necesario para su implementación (traducción de lenguaje natural a restricciones formales, validación previa del mensaje, arbitraje del profesional sobre cada propuesta, generación mediante un motor de optimización, validación clínica automatizada, explicación de infactibilidades, estilo clínico del profesional y seguimiento de generaciones asíncronas), así como **cinco nuevos requisitos no funcionales** específicos de la parte de inteligencia artificial (latencia, validación clínica obligatoria, trazabilidad, protección de datos de salud y explicabilidad).

El resultado es un catálogo final de **29 requisitos funcionales** (21 conservados del catálogo inicial, 8 nuevos del copiloto) y **14 requisitos no funcionales** (7 generales, 7 específicos de IA). La reducción global de requisitos funcionales respecto al catálogo original es del 52 %, mientras que la carga de trabajo real no disminuye en la misma proporción ya que cada uno de los ocho requisitos nuevos del copiloto arrastra consigo una arquitectura técnica compleja.

De esta forma se redefine el cómputo de innovaciones clave; pasan de ser ocho inicialmente a centrar el foco en una única innovación principal de mucha mayor ambición técnica: el **copiloto de planes nutricionales per-nutricionista**. Las otras siete pasan a *trabajo futuro* (formularios con reglas de alerta, lista de la compra inteligente, *check-in* gamificado, base de alimentos con reconocimiento por IA, *marketplace*, motor de automatizaciones y comunidad entre clientes).

La tabla 3.1 resume el antes y el después del alcance del TFG.

Categoría	V1 (original)	V2 (final)	Variación
Requisitos funcionales	61	29	−52 %
Requisitos no funcionales	7	14	+7 (+8 nuevos, −1 eliminado)
Innovaciones clave	8	1 (copiloto IA)	−7 a trabajo futuro
Bloques de funcionalidad	13	8 (parcialmente reducidos)	−5 bloques eliminados

Tabla 3.1: Reformulación del alcance del TFG: del catálogo original al definitivo.

3.1.1. Criterio de decisión aplicado en el triaje

Con la solicitud por parte de los nutricionistas como referencia, cada requisito del catálogo original se ha clasificado aplicando el siguiente criterio, en este orden de prioridad:

1. **¿Contribuye al funcionamiento del copiloto IA?** Requisitos como el perfil del cliente con mediciones (RF-01, RF-04, RF-05), la base de datos de alimentos (RF-25, RF-45, RF-48) o la estructura básica de planes (RF-24, RF-26, RF-27, RF-29) se mantienen porque alimentan directamente el *pipeline* de generación.
2. **¿Contribuye a la evaluación experimental del copiloto?** El *check-in* del cliente se conserva en versión reducida (sólo marcar qué comidas del plan se han completado) porque alimenta la métrica de adherencia, que es una de las variables clave para demostrar empíricamente en la defensa que los planes generados funcionan. Del mismo modo, la gráfica de peso se mantiene por el mismo motivo.
3. **¿Es imprescindible para que el producto tenga sentido como aplicación?** La mensajería mínima (chat RF-12, RF-13) y el calendario reducido (RF-06, RF-09, RF-10, todas las citas en modalidad *online*) se conservan porque sin ellas la plataforma no funciona como herramienta de gestión profesional coherente, aunque no contribuyan directamente al copiloto.
4. **Todo lo demás pasa a trabajo futuro.** Los requisitos que no cumplen ninguno de los tres criterios anteriores, por valiosos que sean como producto, se documentan en el capítulo de conclusiones como extensiones naturales del TFG para una segunda versión post-defensa.

A continuación, se presentan los requisitos finales agrupados por bloque temático en secciones, manteniendo la estructura del catálogo original del Anexo A para facilitar la trazabilidad.

3.1.2. Bloques eliminados completos y su justificación

Cinco bloques completos del catálogo original se eliminan de la implementación del TFG y se documentan como trabajo futuro en el capítulo 8. La tabla 3.2 los recoge con la razón concreta que justifica su exclusión.

Bloque eliminado	RFs afectados	Razón de exclusión
Formularios personalizables	RF-14 a RF-17	Editor visual <i>drag-and-drop</i> con plantillas y asignación programada. Alto coste de desarrollo, sin contribución al copiloto.
Alertas inteligentes	RF-18 a RF-23	Motor de reglas sobre formularios con prioridades y <i>dashboard</i> dedicado. Su concepto se absorbe parcialmente en RF-66 (validación clínica del copiloto).
Lista de la compra	RF-30 a RF-32	Generación desde el plan con sustituciones y marcado. Derivable trivialmente del plan generado cuando sea necesario; no aporta al diferencial IA.
<i>Marketplace</i> de recursos	RF-51 a RF-55	Catálogo, publicación, gestión de ventas y biblioteca personal. Implica un subsistema completo de comercio electrónico sin aporte académico al núcleo IA.
Motor de automatizaciones	RF-41, RF-42, RF-56 a RF-58	Motor de reglas configurable con disparadores y acciones. Alta complejidad; la automatización básica se suprime también al desaparecer los formularios y las alertas que la disparaban.

Tabla 3.2: Bloques completos eliminados del alcance tras la reformulación.

Adicionalmente, se eliminan requisitos individuales de otros bloques: videollamadas y llamadas de voz (RF-43, RF-44), chat entre clientes (RF-61), multi-clínica y colores por tipo de cita (RF-07, RF-08, RF-11), plantillas reutilizables de planes (RF-28), sustitución de comida por parte del cliente (RF-59), gamificación del *check-in* (RF-34, RF-35), *heat map* y fotos de progreso (RF-37, RF-38), exportación a PDF (RF-40), escáner de códigos de barras y reconocimiento fotográfico (RF-46, RF-47), registrar alimento consumido (RF-49) y comparativa con objetivos diarios (RF-50). Todos se justifican individualmente en las secciones de su bloque correspondiente o en el capítulo de trabajo futuro.

La reformulación también operó en sentido inverso. El prototipo original, pese a su amplitud, tenía huecos que la implementación tuvo que cubrir: no existía pantalla de inicio de sesión —la navegación arrancaba directamente en el panel— ni catálogo de alimentos propio del nutricionista, dos piezas imprescindibles en cuanto la aplicación dejó de ser un diseño navegable y pasó a ser un producto con datos reales. Ambas se incorporaron al alcance durante el desarrollo.

Con esta reformulación cerrada, el diseño e implementación del TFG se guiará por las siguientes secciones de requisitos.

3.2. Requisitos no funcionales

Los requisitos no funcionales definen las restricciones y condiciones de calidad que debe cumplir el sistema, independientemente de las funcionalidades concretas que ofrezca. Respecto al catálogo V1 (Anexo A), cinco se conservan sin cambios, uno se reformula y uno se elimina al quedar fuera del alcance las clínicas como entidad. A ellos se suman ocho nuevos: cinco específicos del copiloto que nacen con la reformulación, y tres más (uno general y dos de IA) que se consolidaron durante el desarrollo, cuando decisiones tomadas en las reuniones de dirección y en la revisión externa del diseño demostraron ser en realidad requisitos del sistema.

3.2.1. Requisitos no funcionales generales

RNF-01 — Interfaz diseñada para escritorio. Durante las entrevistas con nutricionistas se constató que tanto los profesionales como sus clientes trabajan desde un portátil durante las consultas. Ninguno de los nutricionistas consultados realizaba gestiones desde el móvil en su entorno laboral. Por este motivo, toda la interfaz se ha diseñado para una resolución de escritorio (1280 px de ancho mínimo), siguiendo un enfoque *desktop-first*. Esto no excluye el acceso desde otros dispositivos, pero la experiencia principal está optimizada para portátil.

RNF-02 — Aplicación web accesible desde navegador. Al tratarse de una aplicación web, no requiere instalación ni configuración local. Esto es especialmente relevante porque muchos nutricionistas trabajan en varias clínicas distintas y no siempre utilizan el mismo equipo. Acceder desde cualquier navegador garantiza disponibilidad sin dependencia del hardware.

RNF-03 — Interfaz moderna, limpia y profesional. La aplicación va a ser utilizada por profesionales de la salud en su entorno de trabajo, por lo que debe transmitir confianza y profesionalidad. Se tomó como referencia visual Nutrium, por su interfaz limpia y orientada a productividad. El objetivo es que el nutricionista perciba la herramienta como un instrumento profesional, no como un producto genérico.

RNF-04 — Sidebar permanente adaptable. Al tratarse de una herramienta de uso diario, el nutricionista necesita acceder rápidamente a cualquier sección sin perder el contexto de lo que está haciendo. NutriApp ofrece un *sidebar* fijo y siempre visible, con dos modos de visualización que el usuario puede cambiar: un modo expandido en el que cada entrada incluye icono y etiqueta de texto, y un modo compacto en el que solo se muestran los iconos para aprovechar más espacio horizontal en pantalla. Este enfoque, inspirado en aplicaciones como Notion, permite conservar la navegación permanente sin penalizar el espacio disponible para el contenido principal.

RNF-05 — Separación clara entre panel de nutricionista y panel de cliente. Ambos perfiles de usuario tienen necesidades completamente distintas: el nutricionista gestiona múltiples clientes, citas y planes; el cliente interactúa con sus propios datos, su plan y su progreso. Mezclar ambas experiencias en una sola interfaz generaría confusión. Por ello, cada perfil dispone de su propio panel con navegación, funcionalidades y diseño adaptados, aunque ambos paneles comparten la misma base de datos.

RNF-07 — Seguridad y privacidad de datos. La plataforma maneja datos de salud de los clientes, que son especialmente sensibles. El sistema debe contemplar autenticación segura, roles diferenciados y acceso restringido a la información según el perfil del usuario. Por ejemplo, las notas privadas que el nutricionista escribe sobre un cliente no deben ser visibles para dicho cliente en ningún caso. Este requisito se complementa con RNF-IA-4 para el tratamiento específico de datos de salud empleados en la generación de planes por IA.

RNF-08 — Aislamiento multi-tenant verificado. Cada nutricionista solo puede ver y modificar sus propios datos, y cada cliente los suyos. Este aislamiento no puede depender de que la interfaz no muestre lo ajeno: debe estar garantizado en la propia capa de datos, de forma que manipular una URL o llamar directamente al API devuelva cero resultados. El requisito exige además verificación empírica: el aislamiento se comprueba con baterías de pruebas automatizadas que intentan lecturas y escrituras cruzadas entre cuentas y confirman que ninguna alcanza datos ajenos. Este requisito se consolidó durante el desarrollo a petición de los tutores del trabajo.

3.2.2. Requisitos no funcionales específicos del copiloto IA

Los siete requisitos siguientes recogen restricciones inherentes a la incorporación del copiloto inteligente en el sistema. Los cinco primeros nacen con la reformulación V2; los dos últimos se consolidaron durante el desarrollo, uno a propuesta de la revisión externa del diseño y otro como decisión de las reuniones de dirección. Todos ellos deben cumplirse antes de que cualquier borrador de plan generado por la IA llegue al nutricionista o, por extensión, al cliente.

RNF-IA-1 — Generación acotada en tiempo y sin bloqueo. La generación de un plan no puede dejar la aplicación inutilizable mientras se calcula. El requisito se concreta en dos garantías: la interfaz responde de inmediato —la generación se ejecuta como tarea en segundo plano y el profesional puede seguir trabajando mientras consulta su estado— y el motor de optimización trabaja con un límite de tiempo explícito, calibrado contra los recursos del entorno de producción, transcurrido el cual devuelve la mejor solución encontrada o un diagnóstico de por qué no existe ninguna.

RNF-IA-2 — Validación clínica obligatoria. Toda generación debe pasar por un validador clínico determinista antes de ser entregada al nutricionista. Ningún bo-

rador que incumpla restricciones duras —alergias declaradas del cliente, alimentos prohibidos, rangos de calorías y macronutrientes fuera de tolerancia, incompatibilidades conocidas con patologías registradas— puede llegar a presentarse. Este requisito es una garantía de seguridad clínica y no es negociable: su cumplimiento es condición previa a cualquier mejora de calidad estética o estilística del plan.

RNF-IA-3 — Trazabilidad completa de cada generación. Cada generación queda trazada extremo a extremo: *prompt* enviado, modelo utilizado y su versión concreta, tokens consumidos, latencia y coste de la llamada, restricciones resultantes de la traducción y veredicto del validador clínico. La motivación es doble: auditoría posterior en caso de incidencia clínica y recogida de datos para la evaluación experimental del sistema.

RNF-IA-4 — Protección de datos de salud conforme al RGPD. El tratamiento de los datos de salud de los clientes debe cumplir lo establecido por el Reglamento General de Protección de Datos para las categorías especiales definidas en su artículo 9. Esto incluye consentimiento explícito para el uso de los datos en la generación por IA, cifrado en tránsito y en reposo, ausencia de retención por parte de los proveedores externos de modelos de lenguaje, y la posibilidad efectiva de ejercer los derechos de acceso, rectificación y supresión.

RNF-IA-5 — Explicabilidad mínima. El sistema debe permitir al nutricionista entender por qué se le propone lo que se le propone. Esto se concreta en tres frentes: las restricciones que gobiernan cada generación son visibles y editables (nada se aplica sin que el profesional lo haya aceptado), cada traducción de lenguaje natural queda registrada con su entrada y su salida, y cuando un plan resulta imposible el sistema señala qué restricciones entran en conflicto en lugar de fallar sin explicación. La motivación académica es garantizar que el copiloto no se comporta como una caja negra, la motivación práctica es que el nutricionista pueda intervenir en el punto exacto en el que el sistema se equivoca.

RNF-IA-6 — Firma profesional obligatoria. Ningún plan generado llega al cliente sin la firma explícita del nutricionista. El sistema puede producir y almacenar tantos borradores como haga falta, pero solo pasará a ser un plan entregable al cliente si el profesional lo autoriza y firma, esa excepción se aplica en la propia capa de datos: para el cliente, los borradores no existen. Este requisito es el que convierte al sistema en un asistente del profesional colegiado y no en un generador autónomo de dietas.

RNF-IA-7 — Independencia del motor respecto al LLM. El motor de generación debe funcionar sin el componente de inteligencia artificial: las restricciones pueden introducirse manualmente por el profesional y el plan resultante es exactamente el mismo. Esta decisión, acordada con los tutores, ordena toda la arquitectura: el modelo de lenguaje es una capa de conveniencia sobre una interfaz estable de restricciones, no una pieza imprescindible, y garantiza que la plataforma siga siendo

útil si el modelo no está disponible o su calidad resulta insuficiente para un caso concreto.

3.3. Requisitos funcionales

Los requisitos funcionales describen las acciones concretas que el sistema debe permitir realizar. Se organizan por bloques temáticos, indicando para cada requisito el actor principal que lo ejecuta: el nutricionista (N), el cliente (C) o el propio sistema de forma automática (S). El copiloto IA constituye la única innovación clave del TFG tras la reformulación y se describe en su propio bloque. Los demás bloques se presentan reducidos al subconjunto de requisitos efectivamente implementados; los requisitos propuestos a trabajo futuro se listan al final del capítulo con referencia al capítulo 8.

3.3.1. Gestión de clientes

La gestión del perfil del cliente es el punto de entrada de información al sistema y, en particular, al copiloto IA: los datos personales, las mediciones antropométricas y las restricciones clínicas y dietéticas registradas aquí son precisamente las variables que condicionan la generación de un plan personalizado. Las notas privadas del nutricionista se mantienen en el alcance porque pueden incluir observaciones sobre el plan del cliente sin añadir complejidad técnica al sistema.

ID	Requisito	Actor
RF-01	Crear, editar y consultar perfiles de clientes con historial clínico, datos personales, objetivos y mediciones.	N
RF-02	Buscar y filtrar clientes por nombre y por estado (activo, inactivo, nuevo).	N
RF-03	Escribir notas privadas asociadas a cada cliente, visibles únicamente para el nutricionista. Pueden incluir observaciones sobre el plan o el seguimiento.	N
RF-04	Editar datos personales y registrar mediciones corporales.	C
RF-05	Consultar las mediciones y la evolución de cada uno de sus clientes.	N
RF-60	Acceder a una ficha detallada por cliente que consolide datos básicos, notas privadas, plan actual, próxima sesión y adherencia reciente en una sola pantalla.	N

Tabla 3.3: Requisitos funcionales — Gestión de clientes.

3.3.2. Calendario y citas online

El módulo de calendario se reduce respecto a V1 para eliminar el modelo de multi-clínica y los recordatorios automáticos. Todas las citas son en modalidad *online*: el nutricionista configura su disponibilidad y el cliente puede solicitar, cancelar o reprogramar sus propias citas. La justificación de este recorte es doble: por un lado,

al desaparecer la entidad *clínica* se simplifica drásticamente el modelo de datos; por otro, los recordatorios automáticos dependían de un motor de notificaciones programadas cuyo único otro uso (RF-41, RF-42) también sale del alcance.

ID	Requisito	Actor
RF-06	Gestionar una agenda de citas con vista diaria, semanal y mensual.	N
RF-09	Configurar la disponibilidad horaria para que los clientes puedan solicitar citas.	N
RF-10	Ver citas programadas, solicitar nuevas según disponibilidad, y cancelar o reprogramar citas existentes.	C

Tabla 3.4: Requisitos funcionales — Calendario y citas online.

3.3.3. Mensajería

La mensajería queda reducida al chat directo entre nutricionista y cliente con soporte de adjuntos. Las videollamadas y llamadas de voz, así como el chat entre clientes del mismo nutricionista, pasan a trabajo futuro. El chat es imprescindible para la coherencia de la plataforma como herramienta de seguimiento profesional, pero no contribuye directamente al copiloto IA.

ID	Requisito	Actor
RF-12	Ofrecer un chat de mensajería directa entre nutricionista y cliente, con historial completo de conversaciones.	N/C
RF-13	Permitir adjuntar archivos (documentos, imágenes, PDFs) en los mensajes.	N/C

Tabla 3.5: Requisitos funcionales — Mensajería.

3.3.4. Planes nutricionales: estructura base

Este bloque recoge la infraestructura básica de planes nutricionales sobre la que opera el copiloto IA del bloque siguiente. Un plan es una estructura de comidas organizada por día, compuesta por ingredientes o recetas tomados de la base de datos de alimentos, con sustituciones definidas por equivalencia nutricional. El cliente consulta su plan en formato semanal. No se incluyen en V2 las plantillas reutilizables (RF-28), dado que el copiloto IA genera planes a medida por cliente y el uso de plantillas manuales deja de tener sentido; tampoco la sustitución por parte del cliente (RF-59), pospuesta para evitar complicar el flujo y su impacto en el análisis de adherencia.

ID	Requisito	Actor
RF-24	Crear planes nutricionales completos organizados por día y por comida (desayuno, comida, cena, <i>snacks</i>).	N
RF-25	Gestionar una base de datos de alimentos con información nutricional (calorías, macronutrientes, categoría).	N
RF-26	Registrar recetas compuestas por ingredientes de la base de datos de alimentos, con información nutricional agregada (sin instrucciones detalladas de preparación en esta versión).	N
RF-27	Definir sustituciones entre alimentos del mismo grupo nutricional con equivalencia de macronutrientes.	N
RF-29	Consultar el plan nutricional actual con vista semanal, desglose por comidas e información nutricional.	C

Tabla 3.6: Requisitos funcionales — Planes nutricionales: estructura base.

3.3.5. Copiloto de planes nutricionales (*innovación clave*)

Este bloque constituye la **única innovación clave** de NutriApp tras la reformulación. El copiloto se describe sobre un principio rector: el sistema propone y el profesional decide. El modelo de lenguaje no genera dietas; traduce las peticiones del nutricionista a restricciones formales de un vocabulario cerrado, y es un motor de optimización determinista quien construye el plan a partir de ellas, de modo que las garantías clínicas nunca dependen de un componente probabilístico. La arquitectura técnica completa se describe en el capítulo 4; los requisitos no funcionales asociados son RNF-IA-1 a RNF-IA-5 (sección 3.2).

ID	Requisito	Actor
RF-62	Describir en lenguaje natural las necesidades dietéticas de un cliente y obtener su traducción a restricciones formales del catálogo del sistema.	N
RF-63	Validar cada mensaje antes de traducirlo —idioma, pertinencia nutricional, completitud y contradicciones— y devolver avisos accionables en lugar de fallos opacos.	S
RF-64	Revisar cada restricción propuesta por la IA y decidir su destino: incorporarla al perfil del cliente, aplicarla solo a la generación en curso o descartarla.	N
RF-65	Generar borradores de plan que satisfagan las restricciones activas mediante un motor de optimización determinista, operativo también sin el componente de IA.	S
RF-66	Auditar cada borrador con un validador clínico determinista e independiente del motor antes de presentarlo al profesional.	S
RF-67	Explicar las generaciones imposibles señalando el subconjunto de restricciones en conflicto, expresado en lenguaje llano con los nombres reales del catálogo.	S
RF-68	Definir el estilo clínico propio del nutricionista como reglas declarativas que se aplican a la generación de todos sus planes.	N
RF-69	Ejecutar las generaciones de forma asíncrona y seguir su estado desde la interfaz, con las tareas fallidas visibles junto a su explicación.	S

Tabla 3.7: Requisitos funcionales — Copiloto de planes nutricionales.

3.3.6. Check-in mínimo de adherencia

El *check-in* diario, planteado en V1 como innovación clave con gamificación, racha de días y hábitos personalizables, se reduce en V2 a un único requisito funcional: el cliente marca diariamente qué comidas del plan ha completado. La razón de mantener esta versión mínima no es estética sino experimental: el porcentaje de adherencia resultante es una de las variables más relevantes para evaluar empíricamente la calidad de los planes generados por el copiloto IA y demostrar ante el tribunal que el sistema produce planes que los clientes efectivamente siguen.

ID	Requisito	Actor
RF-33	Registrar diariamente qué comidas del plan se han completado, para calcular automáticamente el porcentaje de adherencia.	C

Tabla 3.8: Requisitos funcionales — Check-in mínimo de adherencia.

3.3.7. Progreso del cliente

El bloque de seguimiento se reduce a una única vista gráfica que combina la evolución del peso registrado por el cliente y el porcentaje de adherencia acumulado. El *heat map* tipo GitHub y las fotos de progreso se posponen a trabajo futuro: el

primero por depender del *check-in* en su versión gamificada, y las segundas por no aportar valor al copiloto IA.

ID	Requisito	Actor
RF-36	Visualizar la evolución del peso del cliente y del porcentaje de adherencia al plan a lo largo del tiempo.	C

Tabla 3.9: Requisitos funcionales — Progreso del cliente.

3.3.8. Base de datos de alimentos

La base de datos de alimentos se reduce a la búsqueda por nombre y a la posibilidad de registrar alimentos personalizados por parte del nutricionista. Se elimina la integración con Open Food Facts en favor de BEDCA (Base de Datos Española de Composición de Alimentos, de referencia académica en España), y se posponen a trabajo futuro el escaneo de códigos de barras (RF-46), el reconocimiento de alimentos por imagen (RF-47), el registro directo de consumo (RF-49) y la comparativa con objetivos diarios (RF-50). Los alimentos personalizados se conservan porque permiten al nutricionista cubrir los huecos de BEDCA con preparaciones o productos locales, y porque el copiloto IA necesita un catálogo completo cuando genera un plan.

ID	Requisito	Actor
RF-45	Consultar una base de datos de alimentos con buscador por nombre, alimentada por BEDCA.	N/C
RF-48	Añadir alimentos personalizados a la base de datos (recetas propias o productos locales) con sus macronutrientes.	N

Tabla 3.10: Requisitos funcionales — Base de datos de alimentos.

3.3.9. Dashboard de métricas del copiloto

El *dashboard* general del nutricionista original (RF-39 en V1) se reemplaza por un *dashboard* centrado en las métricas del copiloto IA: tasa de aprobación de los borradores al primer intento, *edit distance* media entre el borrador y la versión final entregada, coste acumulado de llamadas a modelos de lenguaje en el mes en curso, número de planes generados y comparativa de adherencia entre clientes con plan generado por IA y clientes con plan elaborado manualmente. Este cambio de enfoque es coherente con la nueva centralidad del copiloto y aporta, por primera vez en el sector, información cuantitativa sobre el rendimiento del sistema de IA frente a la práctica manual.

ID	Requisito	Actor
RF-39	Visualizar un <i>dashboard</i> con métricas del copiloto IA: tasa de aprobación al primer intento, <i>edit distance</i> media, coste acumulado, planes generados y adherencia IA <i>vs.</i> manual.	N

Tabla 3.11: Requisitos funcionales — Dashboard de métricas del copiloto.

3.4. Requisitos pospuestos a trabajo futuro

Los requisitos del catálogo original que no forman parte del alcance implementado se posponen a trabajo futuro y se documentarán con mayor detalle en el capítulo 8. A continuación se enumeran agrupados por el bloque al que pertenecen originalmente, con una breve razón de posposición.

Formularios personalizables (RF-14 a RF-17). Editor visual *drag-and-drop* con plantillas y asignación manual o programada. Se posponen al sustituir el concepto de diagnóstico basado en formularios por la variable más directa que alimenta al copiloto: las mediciones y restricciones clínicas registradas en el perfil del cliente.

Sistema de alertas inteligentes (RF-18 a RF-23). Motor de reglas condicionales sobre las respuestas de los formularios, *dashboard* de gestión con prioridades y respuestas rápidas desde la alerta. Aunque este bloque era una de las innovaciones clave en V1, su núcleo conceptual —evaluación automática y bloqueo ante situaciones anómalas— queda absorbido en el validador clínico del copiloto IA (RF-66), que aplica reglas deterministas sobre cada plan generado.

Lista de la compra (RF-30 a RF-32). Generación automática de lista agregada, sustituciones sobre ingredientes no disponibles y marcado de compras. Su implementación es trivialmente derivable de los planes generados por el copiloto, pero no aporta al núcleo IA y se deja fuera del TFG.

Marketplace de recursos educativos (RF-51 a RF-55). Catálogo, publicación, gestión de ventas y biblioteca personal del cliente. Implica un subsistema completo de comercio electrónico (pagos, inventario, reseñas) sin aporte académico al núcleo IA.

Motor de automatizaciones (RF-41, RF-42, RF-56 a RF-58). Envío programado de formularios, notificaciones genéricas y motor de reglas configurables con disparadores y acciones. Su alta complejidad, unida al hecho de que los bloques de formularios y alertas que disparaban la mayor parte de sus acciones también salen del alcance, justifica su posposición completa.

Ampliación clínica de la ficha del cliente. La revisión externa del diseño con perspectiva clínica propuso ampliar la ficha del cliente con más de veinticinco variables adicionales —patologías codificadas, medicación, bioquímica, hábitos de vida—, que permitirían afinar tanto la generación como la validación de los planes. Se registra aquí como requisito identificado y se pospone de forma consciente: el modelo de datos actual cubre las variables que el motor consume hoy, y ampliar la ficha sin que ninguna pieza del sistema explote esos datos añadiría coste sin aportar valor.

Requisitos individuales de otros bloques. La tabla 3.12 recoge los requisitos sueltos pospuestos y la razón concreta de cada uno.

ID	Requisito	Razón de posposición
RF-07	Colores por tipo de cita.	No aplica: todas las citas son <i>online</i> .
RF-08	Multi-clínica.	Se elimina la clínica como entidad; arquitectura <i>multi-tenant</i> por nutricionista.
RF-11	Recordatorios automáticos de citas.	Depende del motor de automatizaciones, fuera del alcance.
RF-28	Plantillas de plan reutilizables.	El copiloto IA sustituye el uso manual de plantillas.
RF-34	Racha de <i>check-in</i> gamificado.	La versión mínima de <i>check-in</i> mide adherencia, no gamifica.
RF-35	Hábitos personalizables por el nutricionista.	Idem.
RF-37	<i>Heat map</i> de cumplimiento.	Depende del <i>check-in</i> completo.
RF-38	Fotos de progreso.	No contribuye al copiloto.
RF-40	Exportación a PDF de informes.	No contribuye al copiloto.
RF-43, RF-44	Videollamadas y llamadas de voz.	Requieren integración con proveedores externos; alto coste por bajo valor añadido al TFG.
RF-46	Escáner de códigos de barras.	Integración externa sin aporte al copiloto.
RF-47	Reconocimiento de alimentos por foto con IA.	Línea de IA paralela fuera del foco del TFG.
RF-49	Registrar alimento consumido.	Depende del <i>check-in</i> completo.
RF-50	Comparar alimento con objetivos diarios.	Idem.
RF-59	Sustitución de comida por el cliente.	Complica el flujo y contamina la métrica de adherencia.
RF-61	Chat entre clientes del mismo nutricionista.	Subsistema de mensajería peer-to-peer fuera del alcance.

Tabla 3.12: Requisitos individuales pospuestos a trabajo futuro.

3.5. Dependencias funcionales

Los requisitos del alcance V2 no actúan de forma aislada. La tabla 3.13 recoge las funcionalidades finales de NutriApp y los requisitos implicados en cada una. La

dependencia más relevante es la del **copiloto IA**, que articula a su alrededor la mayoría del resto de requisitos funcionales mantenidos.

Funcionalidad	Requisitos involucrados
Gestión integral del cliente	RF-01, RF-02, RF-03, RF-04, RF-05, RF-60
Agenda de citas <i>online</i>	RF-06, RF-09, RF-10
Mensajería profesional	RF-12, RF-13
Estructura base de planes nutricionales	RF-24, RF-25, RF-26, RF-27, RF-29
Base de datos de alimentos (BEDCA + personalizada)	RF-25, RF-45, RF-48
Copiloto IA de planes*	RF-01 + RF-24 + RF-25 + RF-27 + RF-45 + RF-48 + RF-62 a RF-69
Seguimiento de adherencia	RF-29, RF-33, RF-36
<i>Dashboard</i> del nutricionista	RF-39

Tabla 3.13: Dependencias funcionales de NutriApp V2. La funcionalidad marcada con * es la única innovación clave del TFG.

La lectura cualitativa de la tabla es directa: todos los bloques del alcance convergen en el copiloto IA, ya sea como fuente de datos (perfil, mediciones, base de alimentos, plan base), como mecanismo de entrega (estructura de plan, vista del cliente) o como instrumento de medición de su calidad (adherencia, *dashboard*). La estructura aplanada respecto a V1 es consecuencia directa de haber eliminado las funcionalidades autónomas que no contribuían al copiloto.

3.6. Funcionalidades resultantes

A partir de los requisitos V2 definidos y sus dependencias, las funcionalidades finales de NutriApp se agrupan en las siguientes áreas. La única etiqueta de *innovación clave* se reserva para el copiloto IA; el resto son funcionalidades base necesarias para la coherencia de la plataforma.

Gestión integral del cliente (RF-01 a RF-05, RF-60). Perfiles con historial clínico, mediciones y objetivos; búsqueda y filtrado por nombre y estado; notas privadas del nutricionista; consulta de la evolución; ficha detallada por cliente. Sustituye la combinación de cuaderno físico, notas del móvil y plataforma de la clínica.

Agenda de citas online (RF-06, RF-09, RF-10). Vistas diaria, semanal y mensual; configuración de disponibilidad horaria por el nutricionista; solicitud, cancelación y reprogramación de citas por parte del cliente. Todas las citas son en modalidad *online*.

Mensajería profesional (RF-12, RF-13). Chat directo nutricionista-cliente con historial completo y adjuntos.

Estructura base de planes nutricionales (RF-24 a RF-27, RF-29). Constructor de planes organizados por día y por comida, base de alimentos con información nutricional oficial (BEDCA), registro de recetas como combinación de ingredientes, sistema de sustituciones por equivalencia de macronutrientes, y vista semanal del plan para el cliente. Este bloque define la estructura de datos sobre la que opera el copiloto IA.

Copiloto de planes nutricionales (RF-62 a RF-69). *Innovación clave*. Sistema de generación semi-automática de planes: el nutricionista describe lo que necesita en su idioma, la inteligencia artificial lo traduce a restricciones formales, un motor de optimización construye el borrador y un validador clínico independiente lo audita. El profesional arbitra cada restricción propuesta y siempre aprueba el plan antes de entregarlo; su estilo clínico, declarado una vez, se aplica a todas sus generaciones. La arquitectura técnica se describe en el capítulo 4.

Base de datos de alimentos (RF-45, RF-48). Búsqueda por nombre sobre BEDCA y extensión con alimentos personalizados por el nutricionista (recetas propias, productos locales). Sirve de catálogo tanto para el constructor manual de planes como para el copiloto IA.

Check-in mínimo de adherencia (RF-33). El cliente marca qué comidas del plan ha completado cada día. El sistema calcula automáticamente el porcentaje de adherencia, que es la métrica principal con la que se evaluará la calidad de los planes generados por el copiloto.

Progreso del cliente (RF-36). Gráfico de evolución combinado de peso y porcentaje de adherencia a lo largo del tiempo.

Dashboard de métricas del copiloto (RF-39). Panel del nutricionista con las métricas de rendimiento del copiloto: tasa de aprobación al primer intento, *edit distance* media entre borrador y versión final, coste económico de las llamadas a modelos de lenguaje, número de planes generados y comparativa de adherencia de los clientes con plan IA frente a los clientes con plan manual.

Capítulo 4

Diseño del sistema

Este capítulo describe el diseño del sistema: la arquitectura que conecta sus piezas, el modelo de datos, la seguridad que aísla a cada usuario, la capa de servicio que persiste y firma los planes, el despliegue y la manera de verificar cada parte. Los dos componentes con más peso propio, el motor de generación y el copiloto de IA, tienen sus capítulos aparte (5 y 6); aquí se explica la aplicación sobre la que trabajan. Cada tecnología se justifica con los motivos por los que se eligió en la capa donde se usa.

La sección 4.1 muestra la arquitectura y la decisión que ordena todo el diseño: una tabla de restricciones como interfaz entre quien las produce y quien las consume. Las siguientes cuatro secciones explican las capas una a una, desde los datos al despliegue, y la sección 4.6 muestra cómo se verifica cada bloque de trabajo. Las pruebas y sus resultados se enseñan en el capítulo 7.

4.1. Arquitectura general

La aplicación tiene tres piezas principales: un frontend web con Next.js, una base de datos PostgreSQL gestionada a través de Supabase y un backend propio en Python. Además, se suma el worker de generación, el proceso que ejecuta los trabajos de IA. Durante el desarrollo corre en mi propio ordenador, porque ahí vive el modelo de lenguaje local; la sección 4.5 explica ese reparto y sus alternativas. La figura 4.1 enseña el conjunto y cómo se comunican las piezas.

El reparto lo acordé con los tutores antes de empezar a programar, con un criterio y orden: cada pieza debía ser la opción más simple que soportara los requisitos del capítulo 3, y la construcción iría de los datos hacia la interfaz (primero la base de datos, después su conexión y luego las pantallas), para que cada capa se apoyara sobre una anterior ya verificada.

El backend de generación es un servicio propio con FastAPI. La razón principal es que tanto el solver del capítulo 5 como el ecosistema de acceso a modelos de lenguaje viven de forma natural en Python; y también porque el solver necesita un proceso con límites de tiempo largos, lo cual encaja mal en las funciones *serverless* de las plataformas donde se despliega el frontend. Además, este servicio es el único

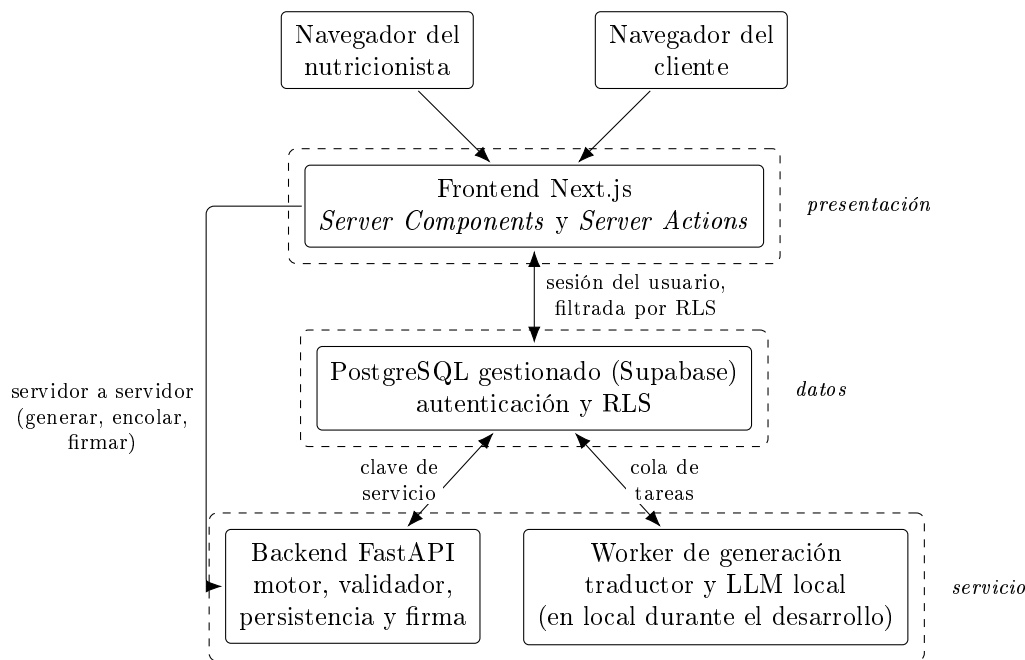


Figura 4.1: Arquitectura del sistema: la capa de presentación, la de datos y la de servicio, con el worker de generación como pieza que corre en un ordenador local.

que guarda la clave privilegiada de la base de datos, que nunca viaja al navegador (sección 4.3).

La capa de presentación usa Next.js con *Server Components*: las páginas se montan en el servidor y leen directamente de la base de datos bajo la sesión del usuario, con las políticas de seguridad de la sección 4.3 como filtro. Así desaparece la capa de API intermedia para casi toda la aplicación. Las escrituras sencillas (dar de alta un alimento, añadir una restricción) siguen el mismo camino con *Server Actions*, funciones que corren en el servidor y validan la entrada antes de tocar los datos. Las operaciones que exigen más garantías como generar, encolar y firmar planes se delegan en el backend con una llamada de servidor a servidor autenticada por un secreto compartido, como se detalla en la sección 4.4.

Para el sistema de diseño seguí la recomendación de Pablo: Radix UI, como base de componentes accesibles sin estilos propios, y Tailwind CSS para aplicarles la paleta y tipografía del prototipo de Figma que servía de referencia.

Por encima del reparto en capas hay una decisión que ordena todo el sistema: las restricciones dietéticas viven en una tabla propia, **diet_constraint**, con un vocabulario cerrado de 16 tipos, y esa tabla es la interfaz entre quien produce las restricciones (el formulario de la ficha del cliente, el formulario del estilo clínico del nutricionista o el traductor de texto libre del capítulo 6) y quien las consume (el motor de generación). La figura 4.2 recoge el esquema.

Este vocabulario se concluyó antes de integrar ningún modelo de lenguaje: primero se construyó y verificó el motor con restricciones escritas a mano, y luego se integró la capa de IA. La vía manual quedó como respaldo: si el traductor falla o el profesional prefiere escribir la restricción él mismo, el formulario alimenta al motor

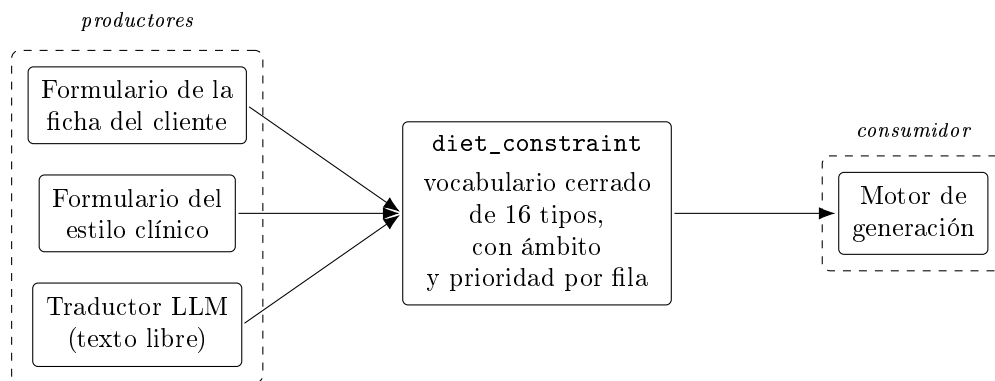


Figura 4.2: La tabla de restricciones como interfaz: tres productores escriben filas del mismo vocabulario y un único consumidor las aplica sin distinguir su origen.

exactamente igual. Los 16 tipos del vocabulario se describen en la sección 5.4.

Del resto de decisiones de la capa de presentación, las que más condicionaron el desarrollo se resumen en cuatro:

- Un único inicio de sesión para los dos roles: la aplicación lleva a cada usuario a su panel, y el panel del cliente vive bajo su propio prefijo de rutas, con guardas equivalentes en cada sección.
- Los 13 tipos de restricción se pueden dar de alta desde la interfaz mediante un único formulario que sirve en tres contextos distintos (la ficha del cliente, el estilo clínico y la pantalla de generación) cambiando sus propiedades.
- Las pantallas que esperan datos nuevos se actualizan por sondeo (cada 15 segundos en la mensajería, cada 4 durante una generación), no es necesario gastar recursos en mantener una conexión abierta, esos tiempos son bastante aceptables.
- Los totales nutricionales de un plan se calculan en la presentación, agregando el modelo alimento-nutriente de la sección 4.2 y avisando cuando falta algún dato de composición.

4.2. Capa de datos

La base de datos es PostgreSQL gestionado a través de Supabase. Fue la recomendación que me dieron mis tutores, y además cumplía con las tres necesidades del catálogo de requisitos: autenticación integrada para resolver los dos roles de la aplicación (RNF-05), políticas de seguridad a nivel de fila para aislar los datos de cada nutricionista en la propia base de datos (RNF-08), y alojamiento en la Unión Europea (la instancia está en Fráncfort), coherente con la protección de datos de salud (RNF-IA-4).

El esquema tiene 22 tablas. La figura 4.3 enseña las que soportan el núcleo de la aplicación, con el resto agrupado por dominios: la agenda y la mensajería, el

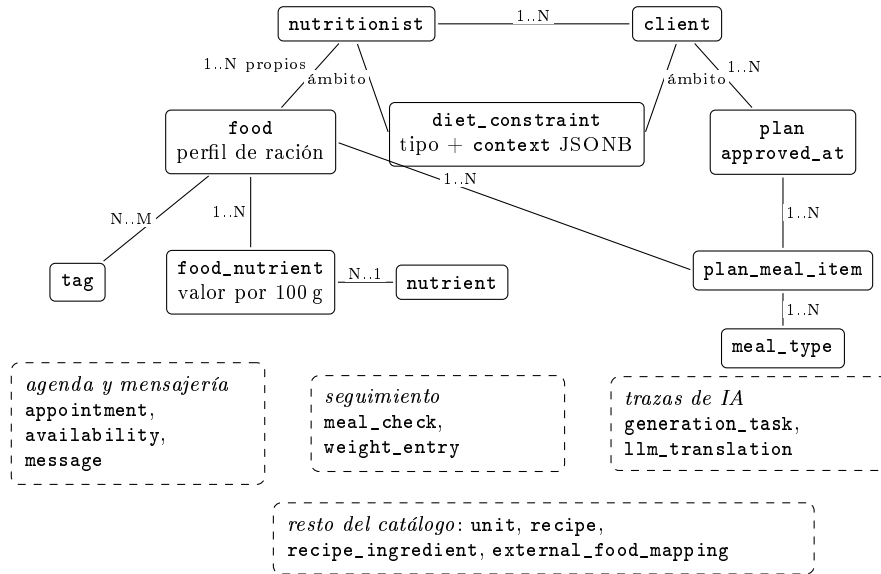


Figura 4.3: Modelo de datos: las tablas del núcleo con sus relaciones y el resto del esquema agrupado por dominios. Las restricciones apuntan además a su objetivo (un alimento, una etiqueta o un nutriente).

seguimiento que registra el cliente y las trazas de los trabajos de IA. El esquema describe el diseño final (el actual), tras todas las decisiones que fueron moldeando el proyecto.

La composición nutricional vive en una tabla propia, **food_nutrient**, con una fila por alimento y nutriente (el valor por cada 100 g), en vez de en columnas planas de la tabla de alimentos. Así, añadir un nutriente nuevo es una fila más en **nutrient**, sin tocar el esquema, a cambio de agregar al leer los totales.

Los planes son dos tablas: **plan** y **plan_meal_item**, con una fila por alimento en cada comida de cada día. No hay tabla intermedia de comidas: la jerarquía días-comidas-alimentos se reconstruye al presentar. Tampoco hay columna de estado: un plan es borrador hasta que tiene fecha de firma (**approved_at**), y esa fecha la escribe solo la capa de servicio (sección 4.4).

El catálogo global y los alimentos propios de cada nutricionista comparten la tabla **food**: un alimento con dueño es privado y uno sin dueño es de todos. Y los argumentos heterogéneos de las restricciones (la cantidad y unidad de un límite, la etiqueta de una exclusión) van en una columna JSONB dentro del esquema fijo de **diet_constraint**: los 16 tipos comparten tabla y difieren en el contenido de esa columna, que la aplicación valida antes de escribir.

Del resto de decisiones de modelado, las específicas de esta aplicación:

- Claves primarias naturales compuestas donde el propio esquema debe impedir duplicados: una marca de comida por plan, día y comida, y un peso por cliente y día.
- La adherencia no se guarda: se calcula a partir de las marcas al leerla, con el mismo criterio en los dos paneles.

- El vínculo entre la ficha del cliente y su cuenta de acceso admite nullos y se rompe sin borrar la ficha si la cuenta desaparece; un disparador crea la fila de perfil que toca al registrarse una cuenta, según su rol.
- Las familias de alimentos son etiquetas, con el dato ausente permitido: un alimento sin clasificar no rompe nada, simplemente no participa en las reglas que dependen de su familia.

El capítulo 5 distingue dos familias de restricciones, y cada una vive en un sitio distinto de la base de datos. El vocabulario clínico son las filas de `diet_constraint` y su enumeración de 16 tipos: crece con una migración del esquema. La plausibilidad alimentaria son datos del catálogo: las etiquetas de cada alimento (familias, alérgenos, franjas) y su perfil de ración (gramos mínimos y máximos por aparición, gramos por pieza). Dar de alta un alimento con su perfil y sus etiquetas amplía lo que el motor sabe hacer sin tocar ni el esquema ni el código.

La capa de acceso desde el frontend impone dos límites que condicionaron el diseño. Las mutaciones desde el navegador van fila a fila, sin transacción que las agrupe: el alta de un alimento con su composición se hace en dos pasos y, si el segundo falla, el primero se deshace con un borrado de compensación. Y las lecturas cortan a un número máximo de filas por consulta, así que los totales de macros se agregan plan a plan en vez de pedirse de golpe. Las operaciones que no toleran estos límites se llevan al backend, que escribe en transacción (sección 4.4).

4.3. Capa de seguridad

La seguridad tiene tres líneas. La sesión viaja en una cookie e identifica al usuario en cada petición. Las guardas de cada sección comprueban el rol y redirigen si no corresponde. Y la última palabra la tiene la base de datos: cada tabla lleva políticas de fila (RLS) que filtran por el usuario de la sesión, así que forzar la URL de un cliente ajeno devuelve cero filas; el filtro lo aplica la propia base de datos, no la pantalla.

Los dos roles conviven sobre las mismas tablas solo con políticas. Como las políticas permisivas se combinan con O lógico, el panel del cliente se añadió escribiendo sus políticas de lectura sin tocar las del nutricionista. Cuando una operación mezcla columnas del actor con columnas que no le pertenecen (el cliente marca una comida, pero el plan es del nutricionista), la política no basta: la escritura pasa por una función con privilegios del sistema que valida la fila y escribe solo lo permitido.

Dos criterios atraviesan todas las políticas. El primero es qué dato puede autorizar: nunca uno que el propio usuario pueda editar; el vínculo entre cuenta y ficha vive en una tabla que solo escribe el servidor. El segundo es que la política también encierra reglas de negocio: el cliente no puede marcar comidas de días futuros ni de comidas que no están en su plan, y los campos que dicen quién envió un mensaje los fija el servidor.

La firma es además una frontera de visibilidad: para el cliente solo existen los planes firmados. Esa regla vive en la política de lectura de `plan`, así que un borrador

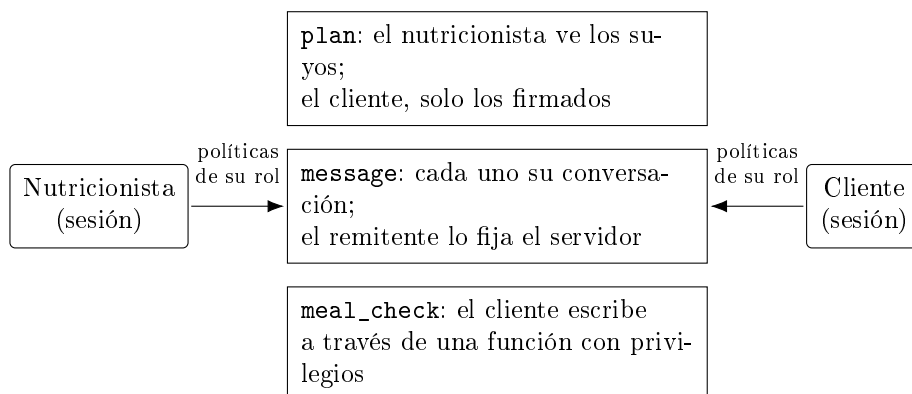


Figura 4.4: Dos roles sobre las mismas tablas: cada uno entra con su sesión y ve o escribe lo que sus políticas permiten, con tres tablas como ejemplo.

es invisible para el cliente aunque ninguna pantalla lo compruebe. La figura 4.4 resume el reparto.

Las claves siguen el mismo reparto: la clave pública que viaja al navegador solo puede lo que la RLS permite, y la clave de servicio, que se salta las políticas, no sale del backend. Aun operando con privilegios, el backend vuelve a comprobar que el cliente pertenece al nutricionista antes de generar o firmar. Cada bloque de desarrollo que tocó la seguridad cerró con una tanda de comprobaciones multi-actor (leer y escribir como un nutricionista, como otro y como un cliente), acumuladas en un banco que se ejecuta con un comando; sus resultados se enseñan en el capítulo 7.

4.4. Capa de servicio

La capa de servicio son el backend FastAPI y el worker de generación. La API es pequeña: generar un plan, encolar un trabajo de IA y firmar. Cada endpoint es una capa fina sobre las funciones puras de los capítulos 5 y 6: lee los datos del cliente, llama a la función y persiste el resultado. La respuesta distingue los dos casos del motor (un plan con sus métricas, o el conflicto explicado), y el frontend la enseña tal cual.

La persistencia es el motivo de que estas operaciones no usen la vía del navegador. Guardar un plan generado son decenas de filas en dos tablas, y aquí sí hay transacción: o se escriben todas o ninguna, mientras que las mutaciones del frontend van fila a fila (sección 4.2).

Firmar es la operación con más peso de diseño y la más simple de código: poner la fecha en `approved_at`. Antes de hacerlo, el endpoint vuelve a validar el plan y la propiedad del cliente; como esa fecha solo la escribe el backend, un plan firmado siempre pasó por la re-validación. Con la política de visibilidad de la sección 4.3, firmar es el acto con el que el profesional convierte su borrador en el plan que el cliente ve: la regla de que el sistema propone y el profesional decide (capítulo 3) queda escrita en la capa de datos.

Los trabajos de IA no se ejecutan durante la petición: la API deja una fila en

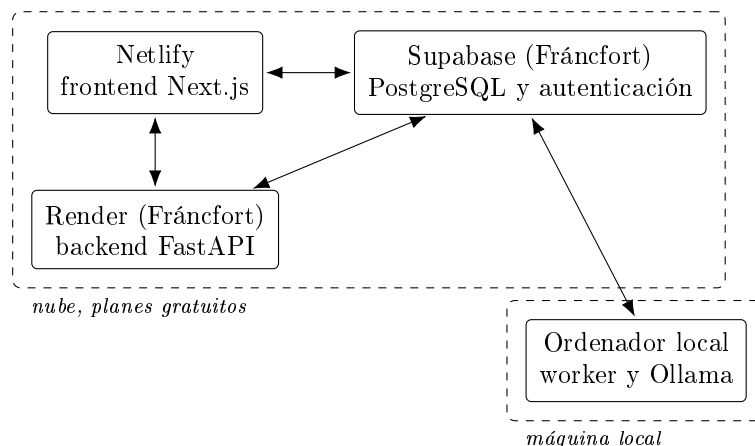


Figura 4.5: Despliegue: tres patas en planes gratuitos de nube y el worker en una máquina local, todas comunicadas a través de la base de datos o de llamadas de servidor a servidor.

la cola `generation_task` y responde al momento con su identificador. El worker reclama tareas pendientes con un bloqueo que impide que dos procesos cojan la misma (en SQL, `FOR UPDATE SKIP LOCKED`), ejecuta el mismo pipeline que el camino síncrono y escribe el resultado en la propia fila; el frontend consulta el estado por sondeo bajo RLS. Se usó la base de datos como cola para evitar una pieza de infraestructura nueva, y el estado de cada tarea (en cola, en curso, terminada o fallida) se consulta como cualquier otro dato.

4.5. Despliegue

El sistema desplegado tiene cuatro patas: el frontend en Netlify, la base de datos en Supabase (Fráncfort), el backend en Render (también Fráncfort) y el worker de generación en un ordenador local. Todo el alojamiento es gratuito. La figura 4.5 enseña el reparto.

El worker corre en local por el modelo de lenguaje. La decisión de privacidad del capítulo 6 fija que el dato clínico no sale hacia ningún proveedor de IA, así que el modelo se ejecuta con Ollama en una máquina propia; y como el alojamiento gratuito no puede correr un modelo de lenguaje, esa máquina es, durante el desarrollo y la evaluación, mi propio ordenador, que hace como si fuese el ordenador del profesional. Si el worker está apagado, la aplicación funciona entera y las tareas de IA esperan en la cola. En un despliegue real la misma pieza podría correr en un servidor de la clínica o cambiar a un motor remoto con garantías contractuales; queda como trabajo futuro en el capítulo 8.

Los límites del alojamiento gratuito están asumidos y medidos. El plan gratuito de Render suspende el backend tras unos minutos sin tráfico: no afecta a una sesión de trabajo normal, pero la primera petición tras una pausa larga paga el arranque en frío. Por su parte, la base de datos gratuita de Supabase se pausa si el proyecto pasa una semana entera sin uso, y hay que reactivarla desde su panel. El límite

de tiempo del solver se calibró contra la CPU de producción, bastante más lenta que la de desarrollo. Y el paralelismo del solver se configura por entorno, porque la memoria del plan gratuito no soporta la configuración de desarrollo (ocho hilos en local, dos en producción). Las medidas se enseñan en el capítulo 7.

Como el prototipo es público, lleva mitigaciones acordes a su carácter académico: un aviso de uso académico en la propia interfaz, datos de demostración con seudónimos y las credenciales fuera del repositorio. El repositorio es privado y sin protección de rama, una limitación asumida para un proyecto de una persona.

4.6. Validación y testing

El desarrollo fue iterativo por bloques (la base de datos, su conexión, el frontend, el motor, la capa de IA, el panel del cliente), y cada bloque cerró con su propia verificación. Esas verificaciones quedan como bancos de pruebas permanentes que se re-ejecutan al cerrar los bloques siguientes, para detectar regresiones al seguir construyendo.

Hay tres familias de verificación. Los bancos deterministas del motor y del validador, con casos contruidos a mano y veredictos esperados. Los runners de seguridad multi-actor, que entran como dos nutricionistas y un cliente y comprueban lectura y escritura tabla a tabla (sección 4.3). Y una prueba de extremo a extremo contra el despliegue real, que genera, valida, persiste y firma un plan en producción. Los resultados se enseñan en el capítulo 7.

El prototipo de Figma sirvió de referencia de la interfaz, y el código se fue adaptando a él según se programaba: algunas cosas se quitaron y otras se añadieron sobre la marcha, y a mitad del desarrollo revisé las diferencias acumuladas para poner al día ambos lados. Los cambios relevantes ya quedan justificados al explicar cada decisión; el resto responde a la versión mínima acordada con los tutores: la aplicación solo enseña lo que tiene una tabla detrás.

Quedan limitaciones asumidas: pantallas con recortes frente al diseño, datos de siembra con fechas relativas que caducan, y la deuda de diseño anterior a la reformulación que la auditoría hizo reducir. Se retoman como trabajo futuro en el capítulo 8.

Capítulo 5

El motor de generación

Este capítulo describe el motor de generación de planes: el componente que, a partir del perfil de un cliente y de un conjunto de restricciones, construye un borrador de plan nutricional que las cumple o explica por qué no existe ninguno que pueda cumplirlas. Funciona sin depender de ningún modelo de lenguaje: las restricciones le llegan ya estructuradas, tanto si las escribió el nutricionista en un formulario como si las propuso el traductor del capítulo 6. Esa independencia se acordó con los tutores y ordenó todo el desarrollo: primero un motor que funciona por sí solo, después la capa de IA por encima.

La sección 5.1 presenta brevemente el paradigma en el que se apoya el capítulo, y la sección 5.2 plantea el problema y su formulación. Las secciones 5.3 y 5.4 recorren las dos familias de restricciones: las que el modelo aplica siempre y las que configura el profesional. La sección 5.5 recoge la función objetivo y la estrategia de resolución, y las dos últimas cubren el diagnóstico de infactibilidad y el validador que audita cada plan. El cuerpo del capítulo se queda con las decisiones y las ecuaciones que las sustentan; la formalización completa, con las 15 reglas y los 16 tipos está en el Anexo C. Las pruebas del motor y las medidas de rendimiento se enseñan en el capítulo 7.

5.1. La programación con restricciones

La programación con restricciones es un paradigma declarativo: en lugar de programar los pasos que construyen una solución, se describe qué condiciones debe cumplir. Un problema se plantea como un modelo con tres piezas: unas variables de decisión (las incógnitas), un dominio para cada una (los valores que puede tomar) y un conjunto de restricciones que las relacionan. Un solver genérico explora los dominios buscando una asignación de valores que satisfaga todas las restricciones; el programa no dice cómo buscarla.

Las restricciones pueden ser de dos clases. Las duras definen qué es una solución válida: el solver descarta cualquier asignación que incumpla alguna. Las blandas expresan preferencias, y se agregan en una función objetivo que mide la calidad de cada solución, para que el solver busque, entre las válidas, la mejor.

Para este problema el paradigma tiene una ventaja más: el modelo se construye antes de cada resolución, así que se puede decidir en ese momento qué restricciones entran. El esqueleto es el mismo para todos los clientes, y sobre él cada generación añade las del cliente concreto: sus alergias, su objetivo calórico, sus preferencias. De esa modularidad sale el catálogo de restricciones de la sección 5.4, que el profesional combina sin que haya que tocar el motor.

5.2. Formulación del problema

El problema se puede enunciar en una frase: dados un cliente, una duración en días, un número de comidas por día y un conjunto de restricciones, decidir qué alimentos aparecen en cada comida y en qué cantidad, cumpliendo todas las restricciones obligatorias y acercándose lo máximo posible a las preferencias.

5.2.1. El problema y su contrato

El resultado es un borrador editable, no un plan cerrado: el nutricionista lo revisa, lo ajusta y decide si lo firma. De ese posicionamiento salen tres propiedades que atraviesan todo el modelo: el plan especifica qué restricciones aplicó, el profesional puede reducir las suyas si lo considera necesario, y cuando el problema no tiene solución el motor explica el conflicto en vez de fallar sin más.

En la implementación, el motor es una función pura: recibe todos sus datos como argumentos y no consulta la base de datos ni escribe en ella. Esto permite probarlo de forma aislada, con casos contruidos a mano, antes de integrarlo en la aplicación. La lectura de datos y la persistencia del plan corresponden al capítulo 4.

La salida distingue dos casos. Si existe un plan que cumple todas las restricciones obligatorias, se devuelve el plan (días, comidas y alimentos con sus gramos) junto con las métricas de la resolución y los avisos no bloqueantes que hayan surgido. Si no existe, se devuelve el subconjunto mínimo de restricciones en conflicto acompañado de una sugerencia legible y la lista de restricciones que se pueden reducir (sección 5.6).

Cuatro cosas quedan fuera a propósito. El motor no consulta ningún modelo de lenguaje, y quién produjo las restricciones le resulta indiferente. No valida clínicamente el resultado: esa auditoría es un paso posterior e independiente (sección 5.7). No decide la estructura del plan: el número de días y de comidas son parámetros de entrada. Y no compone recetas ni modela sustituciones entre alimentos: trabaja con alimentos sueltos, y el plan se expresa en alimentos con sus gramos. El estado real de esos dos requisitos se trata en los capítulos 3 y 8.

5.2.2. Elección de tecnología

El enfoque general se acordó con Alejandro, como comenté en el plan de trabajo: un solver de restricciones que construya el plan y un diagnóstico de infactibilidad cuando no exista. Para implementarlo, me indicó que probablemente la mejor opción fuese CP-SAT, el solver de programación con restricciones sobre dominios enteros

de la biblioteca OR-Tools de Google (Perron y Furnon, 2024).

CP-SAT tenía dos ventajas. La primera es de integración: se usa como una biblioteca más de Python, así que el modelo convive con la carga de datos y con el resto del backend en el mismo lenguaje y el mismo proceso, sin un lenguaje de modelado intermedio. La otra es que CP-SAT permite declarar las restricciones como asunciones y, si el problema no tiene solución, señala qué subconjunto de ellas lo hace imposible. De esta forma, se construye el diagnóstico de la sección 5.6, para convertir el fallo del solver en una explicación útil para el profesional.

La notación del resto del capítulo es independiente de la herramienta; la implementación es CP-SAT.

5.2.3. Conjuntos y parámetros

Se definen cuatro conjuntos base:

- $F = \{\text{alimento}_1, \dots, \text{alimento}_n\}$: los alimentos del catálogo visible para la generación (los globales más los propios del nutricionista). Cada alimento lleva su composición nutricional por cada 100 g, un conjunto de etiquetas (familias, alérgenos, roles y franjas) y un perfil de ración con sus gramos mínimos y máximos por aparición.
- $D = \{1, \dots, \text{duration_days}\}$: días del plan.
- $M = \{1, \dots, \text{meals_per_day}\}$: comidas de un día. Cada comida m lleva asociado un código de franja $c(m)$ (desayuno, comida, cena, merienda).
- $N = \{\text{energía}, \text{proteína}, \text{hidratos}, \dots\}$: los códigos de nutriente presentes en el catálogo (grasa, sodio, calcio y el resto de nutrientes cargados).

Internamente, cada elemento de estos conjuntos se corresponde con un entero: el modelo indexa alimentos, días, comidas y nutrientes por posición, y los nombres solo existen fuera del solver. El tamaño del problema queda fijado por $|F| \cdot |D| \cdot |M|$. La duración y el número de comidas son la dimensión del problema, no algo que el solver decida: entran como argumentos de la función.

Antes de construir el modelo se precálculan en Python los parámetros de entrada. Los que hacen falta para seguir el capítulo son:

- $\text{kcal100}(f)$ y $\text{nut100}(f, n)$: energía y valor del nutriente n por 100 g del alimento f , escalados a entero (la escala se explica más abajo). Si el alimento no tiene dato para un nutriente, vale 0.
- min_f y max_f : gramos mínimos y máximos por aparición del alimento, derivados de su perfil de ración (sección 5.3.2).
- $\text{floor}(c)$: suelo calórico diario de seguridad del cliente, y las cotas diarias de proteína derivadas de su peso (sección 5.3.1).

- `miembros(t)`: para una etiqueta *t*, el subconjunto de alimentos que la llevan. Es la pieza con la que las restricciones razonan sobre familias enteras: prohibir la etiqueta `lactose` prohíbe todos sus miembros.
- Los umbrales estructurales del modelo (máximo de alimentos por comida, mínimos de variedad, y los demás que irán apareciendo), todos constantes con nombre en la configuración del solver. El listado completo con sus valores está en el Anexo C.

5.2.4. Variables de decisión

Al plantear las variables consideré dos formulaciones. La primera era una única variable entera de gramos por cada alimento, día y comida, donde el cero significaba que el alimento no está. La segunda, conseguía separar la presencia de la cantidad: una variable booleana que dice si el alimento aparece en esa comida y una entera con sus gramos. Así que elegí la segunda porque gran parte del modelo razona sobre presencia (prohibiciones, variedad, repeticiones) y expresar esas condiciones sobre una booleana es más directo; los gramos solo importan cuando el alimento está presente.

$$\text{ha_consumido}[f, d, m] \in \{0, 1\}, \quad \text{gramos_alimento}[f, d, m] \in \{0, 1, \dots, \max_f\}, \\ \forall f \in F, d \in D, m \in M$$

Las dos familias se enlazan para que los gramos sean cero si el alimento no está, y una ración dentro de su perfil si lo está:

$$\begin{aligned} \text{ha_consumido}[f, d, m] = 1 &\Rightarrow \text{gramos_alimento}[f, d, m] \geq \min_f, \\ \text{ha_consumido}[f, d, m] = 0 &\Rightarrow \text{gramos_alimento}[f, d, m] = 0 \end{aligned}$$

El mínimo del enlace importa más de lo que parece. En la primera versión estaba con un gramo para contar como presente, y el solver lo aprovechaba metiendo alimentos a 1g solo para cumplir la presencia mínima por comida o para sumar variedad barata. Por ello existe un suelo absoluto de presencia de 10g, y desde la capa de plausibilidad el mínimo y el máximo reales de cada alimento son los de su perfil de ración (sección 5.3.2).

Los alimentos que se sirven por piezas (un huevo, un yogur) llevan además una variable entera de unidades que cuantiza sus gramos a piezas o medias piezas. Se presenta con el resto de la capa de plausibilidad, que es donde tiene su papel.

5.2.5. Trabajo en enteros

CP-SAT opera sobre dominios enteros, y mantener todo el modelo en enteros es la forma natural de trabajar con él. Los valores nutricionales, que en las tablas de composición vienen con decimales, se escalan a entero multiplicándolos por una constante $s = 10$ y redondeando. Así, la suma de un nutriente n en un día d ,

$$\text{nut}_{d,n} = \sum_{m \in M} \sum_{f \in F} \text{nut100}(f, n) \cdot \text{gramos_alimento}[f, d, m],$$

representa la cantidad real del nutriente multiplicada por $100 \cdot s$: los coeficientes son por 100 g y no se divide dentro del modelo. Para comparar esa suma con un valor real v (un objetivo calórico, un mínimo de proteína), se lleva v a la misma escala y se comparan dos enteros.

El mismo patrón, multiplicar en cruz en lugar de dividir, resuelve cualquier restricción expresada como porcentaje o cociente; esto aparecerá varias veces en las secciones siguientes. La resolución final, de un gramo y una décima de unidad nutricional, es más que suficiente en cuanto a precisión en la práctica.

5.3. Restricciones del modelo

Las restricciones de esta sección no las configura el profesional: se aplican siempre, a cualquier plan, con independencia de lo que llegue en la entrada. Son dos familias. Las estructurales (R1 a R8) dan al plan su lógica de conjunto: suelos fisiológicos, variedad y reparto. Las de plausibilidad (R9 a R15) llegaron después, al auditar los planes generados, y se ocupan de la composición de cada comida. La subsección 5.3.3 pone la frontera entre estas reglas y las del profesional.

5.3.1. Restricciones estructurales

Todas son duras: acotan el espacio de soluciones, y un plan que las incumpla no existe para el solver. Son ocho:

- **R1. Presencia mínima.** Cada comida contiene al menos un alimento.
- **R2. Tope de alimentos por comida.** Como mucho 4 alimentos distintos por comida.
- **R3. Suelo calórico diario.** La energía de cada día no baja del metabolismo basal del cliente (se desarrolla abajo).
- **R4. Rango humano de proteína.** La proteína diaria se mantiene entre 0,8 y 2,2 g por kilo de peso, pidan lo que pidan las restricciones.
- **R5. Grasa mínima.** La grasa aporta al menos el 15 % de la energía diaria, necesario para absorber las vitaminas liposolubles.
- **R6. No repetir alimento en el día.** Un mismo alimento aparece como mucho 2 veces entre las comidas de un día.
- **R7. Variedad diaria.** Cada día usa al menos 4 alimentos distintos.
- **R8. Variedad semanal.** Cada ventana de 7 días usa al menos 10 alimentos distintos.

El tope de R2 evita comidas con una lista interminable de ingredientes minúsculos. Lo fijé en cuatro buscando el equilibrio entre una comida completa y la complejidad del modelo, porque cada alimento admitido por comida multiplica las combinaciones que el solver explora. El valor concreto es revisable: una ensalada normal lo supera si se cuentan el tomate, la cebolla o la lechuga, y distinguir esos acompañantes para que no consuman el tope queda como trabajo futuro (capítulo 8). Además, sobre esta cota se apoyan los dominios de la función objetivo (sección 5.5).

El suelo calórico de R3 es el metabolismo basal, no el gasto energético total. El basal se estima con la ecuación de Mifflin-St Jeor (Mifflin et al., 1990) a partir del peso w_c (kg), la altura h_c (cm) y la edad a_c del cliente:

$$\text{BMR}(c) = 10 w_c + 6,25 h_c - 5 a_c + \begin{cases} +5 & \text{en hombres} \\ -161 & \text{en mujeres} \end{cases}$$

y el suelo diario es el mayor entre ese basal y un suelo absoluto por sexo (1200 kcal en mujeres, 1500 en hombres). La distinción tiene consecuencia clínica. Al principio usé como suelo el gasto total (el basal por un factor de actividad), y con un cliente activo el suelo quedaba por encima de su objetivo de pérdida de peso, haciéndolo inalcanzable. El gasto total orienta el objetivo calórico que fije el profesional; el suelo que ninguna dieta puede cruzar es el basal, para permitir los planes de déficit. Si faltan los datos antropométricos, se usa el suelo absoluto por sexo y se emite un aviso.

R6 y R7 no estaban en el diseño inicial. Las añadí después de que los tutores probasen el motor ya desplegado y viesan que los planes salían matemáticamente válidos pero irreales: concentraban todo el pollo en un día y rellenaban el resto casi al azar, solo para cumplir los números. La tercera regla de sentido común, repartir un mismo alimento a lo largo del plan, es blanda a propósito y se cuenta con la función objetivo (sección 5.5): si el catálogo aprieta, el solver puede concentrar algo antes que declararse infactible, y se prefiere un plan imperfecto a un fallo. R6 y R7 sí son duras porque ningún plan real las viola. La tabla 5.1 resume su efecto sobre un mismo caso de una semana, generado con y sin las reglas.

	Sin reglas	Con reglas
Máximo de veces de un alimento en un día	3	2
Mínimo de alimentos distintos por día	3	6
Máximo de apariciones de un alimento en el plan	10	5

Tabla 5.1: Efecto de las reglas de sentido común sobre un mismo caso (7 días, 5 comidas).

Las ocho reglas comparten una asunción que conviene dejar explícita: se asumen válidas para la población general, y permitir al profesional anularlas caso a caso es trabajo futuro (capítulo 8). Pablo puso el ejemplo que enseña el límite: una dieta renal de 45 g de proteína diaria para un cliente de 70 kg queda en 0,64 g/kg, por debajo del suelo de R4, y el motor la declararía infactible aunque esté prescrita a conciencia. La filosofía es la misma que la del suelo calórico: estas reglas son seguridad de último recurso, no una recomendación, y por eso se eligieron lo más

inofensivas posible. La modularidad clínica existe en otro plano: las restricciones duras del profesional se incluyen o no en cada resolución según lo que él configure, sin tocar el motor (sección 5.4).

5.3.2. La capa de plausibilidad del catálogo

Las reglas anteriores dan lógica al plan en su conjunto, pero no dicen nada de qué es una ración razonable ni de cómo se compone una comida. Ese hueco lo destapó una auditoría de todos los planes generados hasta entonces (el método y las cifras están en el capítulo 7): en torno a dos de cada tres cantidades servidas caían exactamente en uno de los dos límites globales de gramos del modelo, raciones de relleno de 10 g para cumplir la variedad y raciones de 300 g de un solo alimento como volumen calórico barato. Los límites universales eran una decisión honesta de partida, pero actuaban de referencia para el motor: el modelo no sabía que una ración razonable depende del alimento, y 10 g de fruta o 300 g de aceite eran soluciones legales.

La respuesta fue sustituir los límites globales por un perfil de ración por alimento, que es un dato del catálogo y no una constante del modelo: cada alimento guarda sus gramos mínimos y máximos por aparición y, si se sirve por piezas, sus gramos por unidad. De ahí salen los límites de la formulación:

$$\begin{aligned}\min_f &= \max(\text{round}(\min_serving_g(f)), 10) \\ \max_f &= \text{round}(\max_serving_g(f))\end{aligned}$$

Un alimento sin perfil (típicamente uno dado de alta a mano por un profesional) usa unos valores por defecto de 20 y 250 g, más estrechos que los límites históricos, para que un alimento sin datos se sirva en raciones prudentes.

Con los perfiles llegan las siete reglas de la capa, que continúan la numeración estructural. Las dos primeras ya se han visto: **R9** es el propio enlace presencia-gramos con los límites del perfil, y **R10** es la cuantización por unidades. Los alimentos con gramos por unidad $gpu(f)$ solo admiten piezas y medias piezas,

$$2 \cdot \text{gramos_alimento}[f, d, m] = k[f, d, m] \cdot gpu(f),$$

con k entero contando medias piezas ($k = 3$ son 1,5 piezas), y un pequeño grupo que no se parte (huevo, yogur) solo admite piezas enteras. Los gramos por unidad se eligieron pares para que las medias piezas caigan en gramos enteros. En base de datos el plan sigue guardando gramos; las unidades son a la vez una restricción del modelo y la forma natural de enseñar esos alimentos en pantalla.

Las otras cinco reglas se apoyan en dos vocabularios nuevos del catálogo, montados sobre las etiquetas existentes: los roles (condimento, dulce) y los momentos del día que admite cada alimento.

- **R11. Lista blanca de franjas.** Un alimento etiquetado con momentos del día solo aparece en esas franjas (la merluza no es un desayuno); un alimento sin etiquetas de momento no queda restringido.

- **R12. Composición mínima de las principales.** Desayuno, comida y cena piden al menos 2 alimentos: una manzana a media mañana es un tentempié normal, una comida de un solo alimento no es una comida.
- **R13. El condimento nunca va solo.** Un condimento exige en su comida al menos un alimento que no lo sea.
- **R14. Tope diario de condimentos.** Entre todos los condimentos, como mucho 2 apariciones al día.
- **R15. Dulce y fruta como complemento.** Como mucho una aparición de dulce o fruta por comida; junto con R12, esto impide que la fruta sea el plato único de una comida principal.

Como muestra del patrón, R13 se expresa acotando cada condimento por la suma de sus posibles acompañantes:

$$\text{ha_consumido}[f, d, m] \leq \sum_{f' \in F \setminus \text{COND}} \text{ha_consumido}[f', d, m],$$

$$\forall f \in \text{COND}, d \in D, m \in M$$

donde COND son los alimentos con rol de condimento. Las siete formulaciones completas están en el Anexo C.

Las franjas de un plan las decide el llamante según su número de comidas, con las principales primero: 3 comidas son desayuno, comida y cena; 4 añaden la merienda; 5 la media mañana; 6 la recena. El número de comidas por defecto de la aplicación se bajó de 5 a 4 para reducir las comidas de muy baja energía. Las medidas de este ajuste están en el capítulo 7.

Las siete reglas de la capa son duras, porque una comida implausible no se compensa con nada, no son algo que el profesional pueda reducir (sección 5.6). La propiedad de la capa es que las reglas son fijas pero su alcance es dato: qué alimentos son condimento, qué franjas admite cada uno y qué ración es razonable vive en el catálogo. Dar de alta un alimento nuevo con su perfil y sus etiquetas lo somete a todas las reglas sin tocar el código.

5.3.3. Restricciones clínicas y restricciones de catálogo

Para organizar las restricciones del modelo, se responde a una pregunta de diseño: quién define cada cosa.

Las restricciones clínicas son las del profesional. Viven como filas en la base de datos, en un vocabulario cerrado de 16 tipos (sección 5.4), con un ámbito (un cliente concreto, o el estilo clínico del nutricionista aplicado a todos los suyos) y la prioridad que él decida. Expresan juicio clínico: una alergia, un objetivo calórico, una preferencia de dieta. Se pueden introducir mediante el formulario manual o mediante el traductor del capítulo 6, y el motor las consumirá igualmente.

Las restricciones de catálogo son las del modelo: las estructurales y las de plausibilidad. No las escribe nadie por cliente; se aplican siempre. Expresan lo que cualquiera da por hecho antes de hablar de nutrición: que una comida no es un chorro de aceite ni la merluza un desayuno. Este conocimiento no es vocabulario del profesional, y ponérselo delante sería ruido: ningún nutricionista prescribe que el aceite vaya acompañado, igual que no prescribe que los platos se sirvan en un plato.

La frontera delimita también la responsabilidad ante un plan malo: si viola una clínica, el fallo es del motor y el validador posterior lo cazaría (sección 5.7); si es implausible, lo que falta es una regla o un dato de catálogo. Dónde vive cada mitad en la base de datos y cómo evoluciona se especifica en el capítulo 4.

5.4. El catálogo de restricciones clínicas

Las restricciones del profesional llegan al motor en un vocabulario cerrado de 16 tipos. De hecho, se cerró ese vocabulario antes de tocar ningún modelo de lenguaje para ordenar las dos mitades del sistema: cualquier productor tiene que expresarse en estos tipos, y el motor solo entiende de ellos. Cada fila trae el tipo, su objetivo (un alimento, una etiqueta, un nutriente, un valor), la prioridad que la hace dura o blanda y un peso de 1 a 10 que gradúa su importancia si es blanda.

Dos de los 16 tipos son estructurales (número de comidas y duración del plan): fijan la dimensión del problema y no generan restricción. Los otros 14 se agrupan en objetivos de energía y macros, límites de nutrientes, prohibiciones y preferencias, reparto y frecuencia, variedad y combinaciones. La tabla 5.2 los resume; las formulaciones completas están en el Anexo C.

Tipo	Familia	Por defecto	Patrón
meals_per_day	estructural	estructural	dimensiona las comidas
plan_duration_days	estructural	estructural	dimensiona los días
kcal_target	energía	blanda	desviación absoluta
macro_target	macros	blanda	desviación absoluta
nutrient_min	nutrientes	configurable	suma $\geq$ / defecto blando
nutrient_max	nutrientes	configurable	suma $\leq$ / exceso blando
nutrient_ratio	nutrientes	configurable	multiplicación cruzada
forbid_food	prohibición	dura	fija a cero
forbid_tag	prohibición	dura	fija a cero por familia
prefer_food	preferencia	blanda	bonifica en el objetivo
prefer_tag	preferencia	blanda	bonifica por familia
meal_kcal_ratio	reparto	blanda	desviación del reparto
max_servings_per_period	frecuencia	configurable	suma en ventana
no_repeat_food	variedad	blanda	ventana por alimento
no_repeat_tag	variedad	blanda	ventana por familia
forbid_combination	combinación	dura	suma por comida ≤ 1

Tabla 5.2: Catálogo de restricciones clínicas: los 16 tipos.

El patrón de cada tipo se ve con dos ejemplos. Una prohibición de etiqueta (el

mecanismo de las alergias e intolerancias) es dura por defecto y fija a cero todas las apariciones de los alimentos afectados:

$$\text{ha_consumido}[f, d, m] = 0, \quad \forall f \in \text{miembros}(t), \quad d \in D, \quad m \in M$$

Un objetivo calórico es blando por defecto: no acota el espacio de soluciones, penaliza en la función objetivo la desviación absoluta de la energía diaria respecto al valor pedido, con la linealización de la sección 5.5. Si el profesional lo marca como duro, pasa a ser una igualdad exacta por día.

Qué es duro y qué es blando se decide en tres niveles. Por la naturaleza del tipo: las prohibiciones son duras por defecto (una alergia es absoluta) y los objetivos y preferencias son blandos (son dianas a las que acercarse). Por la fila: el profesional tiene la última palabra y puede marcar cada restricción como dura o blanda. Y por decisión del modelo: las reglas de catálogo de la sección anterior son duras, salvo el reparto, que es blando para no volver infactible un caso con el catálogo justo.

Hay un límite conocido: la variante blanda del cociente entre nutrientes no se modela, porque penalizar la desviación de un cociente exige linealizar el propio cociente y no una simple multiplicación cruzada. Cuando llega, el motor no la aplica pero emite un aviso, para no silenciar una restricción que el profesional puso.

5.5. Función objetivo y resolución

Las restricciones blandas se agregan en una función objetivo que el solver minimiza: una suma de penalizaciones menos bonificaciones (las preferencias premian, y entran con signo negativo porque se minimiza), más dos términos estructurales de variedad y reparto:

$$\text{mín} \left(\sum_i \text{penalización}_i - \sum_j \text{bonificación}_j + w_{\text{variety}} \cdot P_{\text{variety}} + w_{\text{spread}} \cdot P_{\text{spread}} \right)$$

Acercarse a un objetivo es minimizar $|\text{suma} - \text{objetivo}|$, que no es una expresión lineal. Se linealiza con el patrón estándar de dos variables no negativas de exceso y defecto:

$$\text{suma} - \text{objetivo} = \text{over} - \text{under}, \quad |\text{suma} - \text{objetivo}| = \text{over} + \text{under}$$

Un detalle de este patrón resultó importante en la práctica: las cotas de las variables de exceso y defecto se derivan del catálogo real (el exceso llega como mucho a la cota superior del nutriente menos el objetivo, y el defecto como mucho al objetivo) en vez de usar una holgura fija. Con una cota fija, al crecer el catálogo el rango real de las sumas la superaba y el solver rechazaba el modelo entero; acotar ajustado además mejora la propagación. La cota superior de cada nutriente se apoya en la propia R2: como una comida lleva a lo sumo cuatro alimentos, basta sumar las cuatro mayores aportaciones individuales que el catálogo puede dar (la ración máxima de cada alimento por su densidad), en vez de suponer el catálogo entero

coincidiendo en una comida. El techo resultante es entre uno y dos órdenes de magnitud menor sin excluir ninguna solución; su efecto sobre los tiempos se mide en el capítulo 7.

P_{variety} penaliza suavemente cada alimento del catálogo que no se use en todo el plan, para favorecer usar más alimentos distintos cuando nada más lo decide. P_{spread} es la tercera regla de sentido común de la sección 5.3.1: penaliza que un alimento supere un tope de apariciones en el plan, para que la ingesta se reparta en vez de concentrarse. El tope es proporcional a la duración, 0,6 apariciones por día de plan: en una semana un alimento puede aparecer unas cuatro veces antes de penalizar, frente a las diez que llegaron a verse sin la regla (tabla 5.1).

Cada término lleva un peso efectivo que es el producto de dos niveles: un peso fijo por familia (10 el objetivo calórico, 8 la proteína, 4000 las preferencias; el resto está en el Anexo C) y el peso de 1 a 10 de la fila concreta. Los pesos de las familias no nutricionales llevan incorporado un factor de 1000 que compensa la escala entera de las desviaciones: una desviación nutricional se mide en unidades escaladas (una kcal son 1000 unidades) y una preferencia o un alimento sin usar se cuentan de uno en uno, así que sin ese factor mover una familia estructural entera no compensaba ni una kcal de desviación. El análisis de magnitudes que destapó ese desequilibrio se enseña en el capítulo 7. Esto da una regla de prioridad clara sin exponer al profesional un panel de ajuste global.

El reparto calórico por comidas es la excepción documentada: su linealización por multiplicación cruzada arrastra un factor de 1000 propio, así que cuando el profesional lo activa su término domina la función objetivo. Igualar su escala exigiría renormalizar el término entero, y queda como trabajo futuro (capítulo 8).

El modelo se resuelve con un límite de tiempo de 90 segundos, una brecha relativa del 2 % y un número de hilos de búsqueda configurable por entorno. Los tres parámetros cuentan lo mismo: un borrador editable no necesita el óptimo demostrado. El solver para al llegar al 2 % de la cota inferior y, si el tiempo expira, devuelve la mejor solución factible encontrada, distinguiendo en las métricas el óptimo demostrado del factible. El límite de tiempo está calibrado contra la CPU del alojamiento gratuito donde corre el backend, no contra la máquina de desarrollo. Y los hilos son 8 en desarrollo y 2 en ese alojamiento: cada hilo mantiene su propia copia del estado de búsqueda, y con 8 hilos el proceso superó el límite de memoria de la instancia. Adecuar el paralelismo a los recursos es una decisión de despliegue, no del modelo; el coste asumido es que más casos terminan en factible en vez de en óptimo, con la misma calidad de plan útil. Las medidas están en el capítulo 7.

5.6. Diagnóstico de infactibilidad

Cuando las restricciones duras no admiten solución, el motor no falla sin más: devuelve el subconjunto mínimo de restricciones en conflicto. Se apoya en las asunciones de CP-SAT. Cada restricción clínica dura se envuelve en un literal booleano de activación, se aplica condicionada a ese literal y se declara el literal como asunción. Si el modelo resulta infactible, el solver devuelve el conjunto de literales responsables, que se traducen de vuelta a sus restricciones.

Solo las clínicas duras llevan literal. Las reglas de catálogo se aplican sin condición, así que nunca aparecen en el diagnóstico: el conflicto se explica siempre en términos de cosas que el profesional puso y puede cambiar.

El resultado lleva tres piezas: el núcleo mínimo, una sugerencia redactada a partir de los tipos del núcleo y la lista de restricciones que se pueden reducir. Por ejemplo, un objetivo duro de 900 kcal junto a un mínimo duro de 180 g de proteína devuelve el núcleo con esas dos restricciones: 180 g de proteína aportan unas 720 kcal y, sumando el mínimo del resto de macros, superan las 900. Sobre este diagnóstico se construye después la explicación en lenguaje llano del capítulo 6.

5.7. El validador determinista

El plan factible no se da por bueno solo porque lo diga el solver. Antes de llegar al profesional pasa por un validador independiente: otra función pura que recalcula la nutrición del plan desde los datos del catálogo, sin fiarse de las métricas del solver, y lo audita en cuatro familias de reglas: seguridad fisiológica (suelo calórico, rangos de macros, topes de toxicidad de micronutrientes), consistencia con las restricciones clínicas duras (las re-verifica una a una sobre el plan ya construido), estructura (tamaño de las comidas, mínimos de gramos) y plausibilidad (las mismas garantías de la sección 5.3.2). Son dos motores deterministas que deben coincidir: si el solver tuviera un error, el validador lo cazaría antes de la firma.

Cada hallazgo del validador es un rechazo o un aviso, y el criterio para asignarlos no es clínico, es de coherencia: el validador solo rechaza lo que el solver garantiza (las duras, los suelos, la estructura, la plausibilidad) y avisa de lo que el solver no modela. El caso que fijó el criterio fueron los micronutrientes: el solver no les pone techo, así que el calcio o el hierro de un plan válido oscilan de una generación a otra y a veces cruzan un tope de toxicidad. Con ese tope como rechazo, el veredicto dejaba de ser determinista y ahora pasaba o fallaba según la corrida. Como aviso, la distinción queda donde debe: un plan inseguro se rechaza, un plan mejorable se enseña al profesional con sus avisos, y es él quien decide.

Con el validador se cierra el recorrido del motor: de un perfil y unas restricciones a un borrador auditado, o a una explicación de por qué no puede existir. La evaluación de todo ello (los bancos de casos, los tiempos y el efecto medido de cada capa de reglas) está en el capítulo 7, y cómo se expone el motor como servicio dentro de la aplicación, con su persistencia y la firma del profesional, en el capítulo 4.

Capítulo 6

El copiloto de IA

El copiloto de IA es la capa que permite al nutricionista pedir un plan escribiendo lo que quiere en su propio lenguaje. Sobre el motor determinista del capítulo 5 se añade un modelo de lenguaje que convierte ese texto en restricciones del vocabulario cerrado, y una interfaz que enseña cada paso intermedio. Realmente funciona como un asistente de redacción: la IA propone restricciones, y el profesional decide cuáles se aplican, cuáles se guardan y qué plan llega al cliente. Este capítulo explica qué escribe, qué le devuelve el sistema y qué decisiones toma el copiloto.

La sección 6.1 presenta el flujo completo y los puntos donde decide el profesional. Las tres siguientes desarrollan sus piezas: cómo se convierte el texto en restricciones (sección 6.2), por qué el modelo de lenguaje se ejecuta en local (sección 6.3) y cómo se revisa y arbitra lo que la IA propone (sección 6.4). La sección 6.5 cierra el recorrido desde la petición hasta el plan firmado. La generación asíncrona y la validación se cuentan aquí solo en lo que el profesional percibe, porque sus mecanismos ya están descritos en las secciones 4.4 y 5.7; y las pruebas de los modelos de lenguaje, con sus bancos y mediciones, se enseñan en el capítulo 7.

6.1. El flujo del copiloto

El punto de partida del diseño es que el responsable del plan es el profesional colegiado que lo firma. Por eso el copiloto no genera dietas por su cuenta: redacta restricciones a partir de lo que el nutricionista escribe, se las enseña antes de aplicarlas y no guarda nada sin su decisión. Lo que se le pide es traducir la petición a parámetros, para agilizar el proceso.

El flujo se entiende mejor siguiendo una petición real. El nutricionista elige al cliente y escribe el encargo como lo apuntaría en consulta: "sin gluten, unas 2000 kcal al día y al menos 120 g de proteína". El sistema revisa primero el mensaje y, si detecta que falta un dato, que algo choca con lo ya guardado o que la petición se sale de su ámbito, se lo devuelve con el motivo concreto para que lo corrija. Si el mensaje es viable, el traductor lo convierte en restricciones del vocabulario del capítulo 5 y se las presenta en frases llanas, sin jerga técnica. La figura 6.1 recoge el recorrido completo.

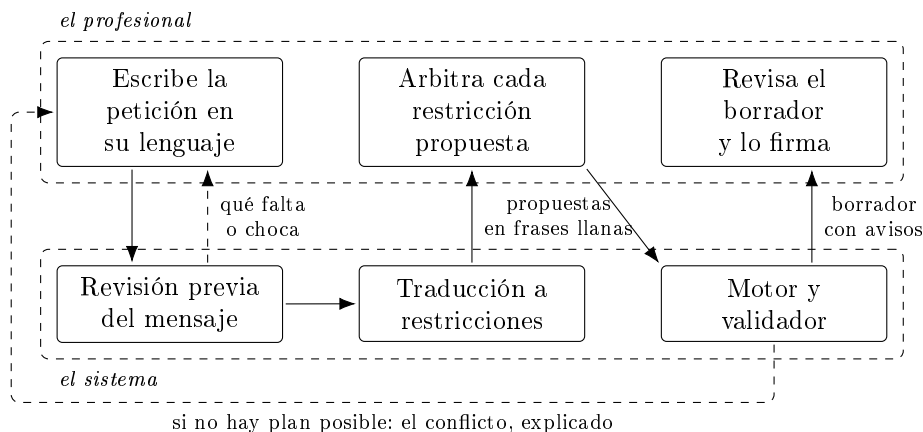


Figura 6.1: El flujo del copiloto. La fila superior son las decisiones del profesional y la inferior el trabajo del sistema: ningún resultado avanza hacia el cliente sin pasar antes por una decisión suya.

Sobre cada restricción propuesta el profesional decide una de tres cosas: guardarla en el perfil del cliente, usarla solo para esta generación o descartarla. Lo que el traductor no entendió también se le enseña con su motivo. Y en la misma pantalla puede añadir restricciones a mano, así que las propuestas de la IA y las suyas conviven en una única lista antes de generar.

Con las restricciones decididas, el motor construye el borrador y el validador lo audita. Si no existe ningún plan que cumpla todo, el conflicto se le explica en una frase con los nombres de su propio catálogo (sección 6.5). El plan generado queda como borrador, y solo pasa a ser visible para el cliente cuando el profesional lo firma, con la política de visibilidad de la sección 4.3.

El copiloto es además opcional en cada paso; la misma pantalla permite saltarlo y fijar las restricciones a mano, y los formularios de la ficha del cliente y del estilo clínico escriben sobre la misma tabla que el traductor (figura 4.2): el motor no distingue el origen. La vía manual queda así como respaldo permanente: si el modelo de lenguaje no está disponible, la aplicación entera funciona igual y el profesional solo pierde la comodidad de escribir en su lenguaje.

6.2. Del texto a las restricciones

El mensaje no va directo al modelo de lenguaje. Antes pasa por una revisión previa: una comprobación determinista que descarta los idiomas que el sistema no soporta, y tres comprobaciones separadas, cada una con una llamada aislada al modelo, que valoran si la petición es del ámbito dietético, si está completa o si se contradice a sí misma. Cuando alguna falla, el profesional recibe el mensaje original con sus detalles precargado para corregirlo sin reescribirlo.

Cada comprobación usa el modelo más pequeño que hace bien ese trabajo. Al medirlas por separado con un banco de mensajes propio, el tamaño necesario resultó distinto en cada una: el modelo que basta para juzgar el ámbito falla la completitud,

y ningún modelo único daba buen resultado en las tres. La matriz de esa medición se enseña en el capítulo 7.

Las contradicciones se reparten entre tres capas, cada problema en la capa más fiable que puede resolverlo. El modelo de lenguaje detecta las categoriales dentro del mensaje, como quitar la carne y a la vez pedir más ternera. También, ya en código y sin pasar por el modelo, se comparan las restricciones propuestas entre sí y con las ya guardadas del cliente, avisando con los nombres reales de lo que choca. Por otro lado, las aritméticas, como un objetivo calórico incompatible con un mínimo de proteína, no las juzga nadie por adelantado: las detecta el motor como infactibilidad y se explican como cuenta la sección 6.5.

Tras la revisión, la traducción se reparte entre el modelo y una capa determinista. El modelo de lenguaje solo extrae la intención: el tipo de restricción, elegido de la lista cerrada de tipos del catálogo que va en el propio prompt, el objetivo (un nutriente o una familia por su código, un alimento por su nombre) y el valor. La capa determinista resuelve después cada código y nombre contra el catálogo real de la base de datos y valida la restricción con las mismas reglas que el formulario manual.

El criterio del reparto es que el modelo elige de un menú, no inventa identificadores, y lo que no se puede resolver se rechaza con su motivo en vez de adivinarse. Si el nutricionista escribe un alimento que no existe en su catálogo, esa petición aparece entre las no aplicadas con su explicación, y el resto del mensaje se traduce igual. Una restricción que supera este camino es idéntica a una creada a mano: mismas columnas, misma validación, mismo consumidor.

Este reparto hace además al traductor poco sensible a cómo esté redactado el mensaje: la lista cerrada y la validación posterior absorben la variación de formato. El estudio que lo midió se enseña en el capítulo 7.

6.3. La elección del modelo de lenguaje

Decidí ejecutar el modelo de lenguaje en local. Es la decisión de privacidad que la sección 4.5 dejaba remitida a este capítulo: los mensajes del profesional hablan de datos clínicos de personas concretas, y uno de los requisitos no funcionales del capítulo 3 fija que ese dato no se envía a ningún proveedor de IA. Con un modelo abierto ejecutado en la propia máquina, el texto no sale del entorno del profesional; la consecuencia sobre el despliegue, que el worker corre donde corre el modelo, ya se contó en la sección 4.5.

El modelo es qwen2.5 (Qwen Team, 2024), una familia abierta con variantes de varios tamaños, y lo ejecuto con el entorno de Ollama, que sirve modelos en local y admite salida con esquema. El traductor usa la variante 7B ajustada a instrucciones; las comprobaciones de la revisión previa usan variantes más pequeñas, elegidas como se contó en la sección 6.2.

A la privacidad se suma el coste: cada consulta a un modelo local cuesta cero, y eso permitió medir, probar y repetir durante todo el desarrollo sin presupuesto. En cambio, con las API gratuitas en la nube, en sus planes sin coste, el proveedor puede usar los mensajes para entrenar, justo lo que el requisito de privacidad prohíbe.

El traductor no depende de esta elección ya que habla con el modelo a través de una interfaz mínima, así que cambiar a un motor remoto con garantías contractuales de no retención sería escribir otra implementación de esa interfaz, sin tocar el resto, por lo que esa puerta queda abierta por diseño, y la comparación entre el motor local y uno remoto queda como trabajo futuro en el capítulo 8. Cada llamada deja además una fila de traza en la base de datos, con la entrada, la salida, el modelo, los tokens y la latencia: información que luego se usará para las mediciones del capítulo 7.

6.4. Revisión y arbitraje

Todas las restricciones que van a alimentar una generación aparecen en la misma pantalla de revisión. Las que propuso el traductor aparecen marcadas como tales, y el profesional puede añadir otras a mano en la misma lista, con el mismo formulario de la ficha del cliente. Sobre cada restricción toma una de las tres decisiones que adelantó la sección 6.1: guardarla en el perfil del cliente (queda en su ficha), aplicarla solo a esta generación (viaja con la tarea y no se guarda) o descartarla.

Los fallos también aparecen aquí. Las peticiones que el traductor no supo aplicar se enseñan con su motivo; los veredictos de la revisión previa, con su aviso, su detalle y el mensaje precargado para corregirlo; y las generaciones fallidas quedan listadas con su explicación.

Los tutores pidieron además que todo esto hablara el idioma del profesional, es decir, que los mensajes mostrados no fuesen confusos o relativos a la base de datos. Por ello, una restricción se enseña siempre como una frase (nunca incluir gluten, unas 2000 kcal al día) y su prioridad como obligatoria o preferible, sin los nombres internos de la base de datos. Las frases salen de una única función compartida por la ficha del cliente, los ajustes y esta pantalla, para que la misma fila se lea igual en toda la aplicación. También el formulario manual se renombró por intención (fijar un objetivo, evitar un alimento o una familia, dar forma a las comidas) en vez de por el nombre técnico de cada tipo.

Además de las restricciones del cliente, el profesional mantiene en sus ajustes sus reglas de estilo: las que aplica a todos sus planes, como no repetir carne roja en días seguidos. Usan el mismo vocabulario y el mismo formulario, y el motor las une con las del cliente en cada generación; es el tercer productor de la figura 4.2. La versión ambiciosa de esta idea, aprender el estilo del profesional a partir de sus correcciones, queda como trabajo futuro en el capítulo 8.

6.5. De la petición al plan firmado

Generar con IA es lento para una petición web: entre el modelo de lenguaje y el solver, una generación tarda desde varios segundos hasta más de un minuto. La primera versión lo hacía todo en la misma petición y bloqueaba la pantalla mientras tanto. Pablo me recomendó encolar el trabajo, responder al instante y consultar el estado periódicamente. La mecánica, con la cola en la base de datos y el worker que la consume, se describió en la sección 4.4.

De esta forma, para el profesional, la aplicación no se queda colgada: confirma la generación, puede irse a otra pantalla y el apartado de planes enseña cada tarea con su estado hasta que el plan aparece. Si el worker no está encendido, la interfaz avisa de que la tarea sigue esperándolo (sección 4.5).

Cuando el motor no encuentra ningún plan, el diagnóstico de la sección 5.6 se convierte en una frase con los nombres reales del catálogo, algo como: el objetivo de 900 kcal choca con el mínimo de 180 g de proteína, y ninguna combinación de los alimentos disponibles cumple los dos. La generación fallida queda en la lista con esa explicación y el detalle del conflicto, y el profesional decide si reduce una de las dos restricciones o cambia el mensaje.

El borrador que sí existe llega auditado por el validador de la sección 5.7, y sus avisos se enseñan junto al plan: un aviso no bloquea, y es el profesional quien decide si el plan vale así. La firma es el último paso y el único que convierte el borrador en el plan que ve el cliente; antes de escribirla, el backend vuelve a validar el plan y la propiedad del cliente (sección 4.4).

Con la firma se cierra el recorrido del capítulo: de una petición escrita en el lenguaje del profesional a un plan auditado que él decidió restricción a restricción. La evaluación de las piezas de IA, con los bancos de las comprobaciones, el estudio de formatos y el comportamiento medido del modelo local, está en el capítulo 7.

Capítulo 7

Evaluación

Capítulo 8

Conclusiones y Trabajo Futuro

Conclusiones del trabajo y líneas de trabajo futuro.

Antes de la entrega de actas de cada convocatoria, en el plazo que se indica en el calendario de los trabajos de fin de grado, el estudiante entregará en el Campus Virtual la versión final de la memoria en PDF.

Introduction

Introduction to the subject area. This chapter contains the translation of Chapter 1.

Conclusions and Future Work

Conclusions and future lines of work. This chapter contains the translation of Chapter 8.

Bibliografía

*Y así, del mucho leer y del poco dormir, se
le secó el cerebro de manera que vino a
perder el juicio.*
(modificar en Cascaras\bibliografia.tex)

Miguel de Cervantes Saavedra

CRONOMETER SOFTWARE INC. Cronometer pro: Nutrition tracking for health professionals. <https://cronometer.com/pro/>, 2025. Consultado en 2025.

HEALTHIE. Healthie: Practice management and patient engagement platform. <https://gethealthie.com>, 2025. Consultado en 2025.

MIFFLIN, M. D., ST JEOR, S. T., HILL, L. A., SCOTT, B. J., DAUGHERTY, S. A. y KOH, Y. O. A new predictive equation for resting energy expenditure in healthy individuals. *The American Journal of Clinical Nutrition*, vol. 51(2), páginas 241–247, 1990.

NUTRIUM. Nutrium: Software de nutrición para profesionales. <https://nutrium.com>, 2025. Consultado en 2025.

OPEN FOOD FACTS. Open food facts: Base de datos colaborativa y abierta de productos alimentarios. <https://world.openfoodfacts.org>, 2025. API REST pública. Consultado en 2025.

PERRON, L. y FURNON, V. Or-tools. <https://developers.google.com/optimization>, 2024. Google.

PRACTICE BETTER. Practice better: All-in-one practice management software. <https://practicebetter.io>, 2025. Consultado en 2025.

QWEN TEAM. Qwen2.5 technical report. <https://arxiv.org/abs/2412.15115>, 2024. Alibaba Group.

Catálogo original de requisitos (versión V1)

Este apéndice recoge el catálogo completo de requisitos *tal y como se formuló inicialmente*, antes del pivote de alcance consensuado con los directores del TFG en abril de 2026. Su función no es normativa —el alcance que realmente se implementa es el descrito en el capítulo 3—, sino *histórica*: conservar la visión inicial como registro del proceso de reformulación del proyecto y facilitar la trazabilidad de cada decisión de recorte.

La versión V1 contenía **61 requisitos funcionales**, **7 requisitos no funcionales** y **8 innovaciones clave**. Tras dos iteraciones de triaje, la versión implementada (V2) reduce estas cifras a 29 requisitos funcionales, 11 requisitos no funcionales (con cinco nuevos específicos del copiloto IA) y una única innovación clave centrada en la generación semi-automática de planes nutricionales por nutricionista. Las razones y el criterio de triaje que llevaron a esta reformulación se desarrollan en la sección 3.1 del cuerpo principal.

Todo el contenido que sigue —tablas de requisitos, dependencias funcionales y funcionalidades resultantes— se presenta sin modificaciones respecto a su formulación original, incluyendo terminología, etiquetas de *innovación clave* y referencias a bloques que posteriormente han sido descartados o redimensionados.

A.1. Requisitos no funcionales (V1)

Los requisitos no funcionales definen las restricciones y condiciones de calidad que debe cumplir el sistema, independientemente de las funcionalidades concretas que ofrezca. Cada uno de ellos se deriva de las necesidades observadas durante la fase de investigación con nutricionistas y del contexto de uso de la aplicación.

RNF-01 — Interfaz diseñada para escritorio. Durante las entrevistas con nutricionistas se constató que tanto los profesionales como sus clientes trabajan desde un portátil durante las consultas. Ninguno de los nutricionistas consultados realizaba gestiones desde el móvil en su entorno laboral. Por este motivo, toda la

interfaz se ha diseñado para una resolución de escritorio (1280 px de ancho mínimo), siguiendo un enfoque *desktop-first*. Esto no excluye el acceso desde otros dispositivos, pero la experiencia principal está optimizada para portátil.

RNF-02 — Aplicación web accesible desde navegador. Al tratarse de una aplicación web, no requiere instalación ni configuración local. Esto es especialmente relevante porque muchos nutricionistas trabajan en varias clínicas distintas y no siempre utilizan el mismo equipo. Acceder desde cualquier navegador garantiza disponibilidad sin dependencia del hardware.

RNF-03 — Interfaz moderna, limpia y profesional. La aplicación va a ser utilizada por profesionales de la salud en su entorno de trabajo, por lo que debe transmitir confianza y profesionalidad. Se tomó como referencia visual Nutrium, el líder del mercado identificado durante el análisis de competencia, por su interfaz limpia y orientada a productividad. El objetivo es que el nutricionista perciba la herramienta como un instrumento profesional, no como un producto genérico.

RNF-04 — Navegación intuitiva con sidebar permanente. Al tratarse de una herramienta de uso diario, el nutricionista necesita acceder rápidamente a cualquier sección sin perder el contexto de lo que está haciendo. Un sidebar fijo y visible en todo momento —similar al de aplicaciones como Gmail o Notion— permite esta navegación inmediata. Se descartó el menú tipo hamburguesa porque en una interfaz de escritorio hay espacio suficiente para mantener la navegación siempre visible.

RNF-05 — Separación clara entre panel de nutricionista y panel de cliente. Ambos perfiles de usuario tienen necesidades completamente distintas: el nutricionista gestiona múltiples clientes, citas, formularios y planes; el cliente interactúa con sus propios datos, su plan y su progreso. Mezclar ambas experiencias en una sola interfaz generaría confusión. Por ello, cada perfil dispone de su propio panel con navegación, funcionalidades y diseño adaptados, aunque ambos paneles comparten la misma base de datos subyacente.

RNF-06 — Escalabilidad. La arquitectura del sistema debe funcionar tanto para un nutricionista autónomo como para una clínica con varios profesionales. Esto afecta directamente al diseño de la base de datos —la relación entre nutricionistas y clínicas se modela como N:M— y a la estructura de permisos, que debe contemplar distintos niveles de acceso según el contexto.

RNF-07 — Seguridad y privacidad de datos. La plataforma maneja datos de salud de los clientes, que son especialmente sensibles. El sistema debe contemplar autenticación segura, roles diferenciados y acceso restringido a la información según el perfil del usuario. Por ejemplo, las notas privadas que el nutricionista escribe sobre un cliente no deben ser visibles para dicho cliente en ningún caso.

A.2. Requisitos funcionales (V1)

Los requisitos funcionales describen las acciones concretas que el sistema debe permitir realizar. Se organizan por bloques temáticos, indicando para cada requisito el actor principal que lo ejecuta: el nutricionista (N), el cliente (C) o el propio sistema de forma automática (S). Aquellos bloques que constituyen innovaciones clave del proyecto —funcionalidades no presentes en ningún competidor analizado— se señalan expresamente.

A.2.1. Gestión de clientes

ID	Requisito	Actor
RF-01	Crear, editar y consultar perfiles de clientes con historial clínico, datos personales, objetivos y mediciones.	N
RF-02	Clasificar y filtrar clientes por estado (activo, inactivo, nuevo) y buscar por nombre u otros criterios.	N
RF-03	Escribir notas privadas asociadas a cada cliente, visibles únicamente para el nutricionista, sustituyendo la agenda física.	N
RF-04	Editar datos personales, registrar mediciones corporales y subir fotos de progreso.	C
RF-05	Consultar las mediciones, fotos de progreso y evolución de cada uno de sus clientes.	N
RF-60	Acceder a una ficha detallada por cliente que consolide foto, datos básicos, notas privadas, plan actual, próxima sesión, alertas recientes e histórico de check-ins en una sola pantalla.	N

Tabla A.1: Requisitos funcionales V1 — Gestión de clientes.

A.2.2. Calendario y citas

ID	Requisito	Actor
RF-06	Gestionar una agenda de citas con vista diaria, semanal y mensual.	N
RF-07	Diferenciar citas por tipo (clínica A, clínica B, privada, online) mediante un código de colores.	N
RF-08	Gestionar múltiples clínicas y asignar citas a cada una de ellas.	N
RF-09	Configurar la disponibilidad horaria para que los clientes puedan solicitar citas.	N
RF-10	Ver citas programadas, solicitar nuevas según disponibilidad, y cancelar o reprogramar citas existentes.	C
RF-11	Enviar recordatorios automáticos de citas próximas.	S

Tabla A.2: Requisitos funcionales V1 — Calendario y citas.

A.2.3. Mensajería

ID	Requisito	Actor
RF-12	Ofrecer un chat de mensajería directa entre nutricionista y cliente, con historial completo de conversaciones.	N/C
RF-13	Permitir adjuntar archivos (documentos, imágenes, PDFs) en los mensajes.	N/C
RF-43	Iniciar una videollamada con el interlocutor directamente desde el chat, sin salir de la plataforma.	N/C
RF-44	Iniciar una llamada de voz con el interlocutor desde el chat.	N/C
RF-61	Ofrecer al cliente un canal de chat con otros clientes del mismo nutricionista, manteniendo el aislamiento entre consultas de distintos profesionales.	C

Tabla A.3: Requisitos funcionales V1 — Mensajería.

A.2.4. Formularios personalizables

ID	Requisito	Actor
RF-14	Crear formularios personalizados mediante un editor visual, con múltiples tipos de campo (texto, número, escala, selección múltiple).	N
RF-15	Gestionar una biblioteca de plantillas de formularios predefinidos (primera consulta, seguimiento, valoración).	N
RF-16	Asignar formularios a clientes de forma manual o programada (semanal, mensual, tras cada cita).	N
RF-17	Ver formularios pendientes, completarlos y consultar el historial de respuestas anteriores.	C

Tabla A.4: Requisitos funcionales V1 — Formularios personalizables.

A.2.5. Sistema de alertas inteligentes (*innovación clave V1*)

Este bloque constituye una de las principales innovaciones de NutriApp en su versión original. Ninguno de los competidores analizados ofrece un sistema de alertas configurables vinculado a las respuestas de formularios. Su objetivo es que el nutricionista pueda actuar de forma proactiva ante problemas de sus clientes, en lugar de enterarse solo cuando el cliente se lo comunica.

ID	Requisito	Actor
RF-18	Definir reglas de alerta condicionales por formulario (ejemplo: “si comidas falladas > 2, generar alerta crítica”).	N
RF-19	Evaluar automáticamente las respuestas de formularios contra las reglas configuradas y generar alertas con nivel de prioridad (crítica, media, informativa).	S
RF-20	Generar alertas automáticas cuando un cliente no realiza check-in durante un número configurable de días.	S
RF-21	Ver, gestionar y resolver alertas desde el dashboard, con indicadores de color por prioridad.	N
RF-22	Resolver alertas y registrar las acciones tomadas al respecto.	N
RF-23	Enviar un mensaje rápido al cliente directamente desde una alerta, con el contexto del caso precargado.	N

Tabla A.5: Requisitos funcionales V1 — Sistema de alertas inteligentes.

A.2.6. Planes nutricionales

ID	Requisito	Actor
RF-24	Crear planes nutricionales completos organizados por día y por comida (desayuno, comida, cena, snacks).	N
RF-25	Gestionar una base de datos de alimentos con información nutricional (calorías, macronutrientes, categoría).	N
RF-26	Crear y gestionar recetas compuestas por alimentos de la base de datos, con instrucciones de preparación e información nutricional agregada.	N
RF-27	Definir sustituciones entre alimentos del mismo grupo nutricional con equivalencia de macronutrientes.	N
RF-28	Guardar planes nutricionales como plantillas reutilizables para aplicar a otros clientes.	N
RF-29	Consultar el plan nutricional actual con vista semanal, desglose por comidas, recetas e información nutricional.	C
RF-59	Sustituir una comida del plan por una opción equivalente pre-configurada por el nutricionista, manteniendo el equilibrio de macronutrientes del día.	C

Tabla A.6: Requisitos funcionales V1 — Planes nutricionales.

A.2.7. Lista de la compra inteligente (*innovación clave V1*)

Otra innovación diferenciadora de NutriApp en su planteamiento original. Los competidores que ofrecen listas de la compra las generan de forma estática; ninguno incorpora un sistema de sustituciones inteligentes basado en equivalencia nutricional.

ID	Requisito	Actor
RF-30	Generar automáticamente una lista de la compra a partir del plan nutricional asignado, organizada por categorías de alimentos.	S
RF-31	Marcar ingredientes como “no disponible” y recibir sugerencias de sustitución del mismo grupo nutricional.	C
RF-32	Marcar ítems como comprados y exportar o compartir la lista.	C

Tabla A.7: Requisitos funcionales V1 — Lista de la compra inteligente.

A.2.8. Check-in diario y gamificación (*innovación clave V1*)

La tercera innovación clave del proyecto en su versión original. Los competidores analizados ofrecen, en el mejor de los casos, check-ins básicos sin mecánicas de gamificación ni conexión con un sistema de alertas.

ID	Requisito	Actor
RF-33	Registrar diariamente: agua consumida, suplementos tomados, comidas completadas del plan, ejercicio realizado y hábitos personalizados.	C
RF-34	Calcular y mostrar la racha de días consecutivos con check-in completado, incluyendo comparación con la mejor racha histórica.	S
RF-35	Configurar hábitos personalizados para cada cliente (horas de sueño, nivel de estrés, meditación, etc.).	N

Tabla A.8: Requisitos funcionales V1 — Check-in diario y gamificación.

A.2.9. Seguimiento y progreso

ID	Requisito	Actor
RF-36	Visualizar la evolución mediante gráficos de peso, medidas corporales y porcentaje de adherencia.	C
RF-37	Mostrar un calendario de cumplimiento tipo <i>heat map</i> (estilo contribuciones de GitHub) basado en los check-ins diarios.	S
RF-38	Subir y comparar fotos de progreso organizadas cronológicamente.	C

Tabla A.9: Requisitos funcionales V1 — Seguimiento y progreso.

A.2.10. Informes y análisis

ID	Requisito	Actor
RF-39	Visualizar un dashboard con métricas globales: clientes activos, citas del día y formularios pendientes.	N
RF-40	Exportar informes de clientes y métricas en formato PDF.	N

Tabla A.10: Requisitos funcionales V1 — Informes y análisis.

A.2.11. Base de datos de alimentos con reconocimiento inteligente (*innovación clave V1*)

Una de las fricciones principales detectadas durante la investigación es el tiempo que tanto el nutricionista como el cliente dedican a consultar información nutricional sobre alimentos concretos. El cliente, sobre todo, suele abandonar el registro de su dieta porque implica buscar manualmente las calorías de cada producto que consume. La propuesta original de NutriApp resolvía este punto mediante una base de datos de alimentos accesible desde ambos paneles, integrada con una API abierta como Open Food Facts, y complementada con dos mecanismos de captura rápida: el escaneo de códigos de barras y el reconocimiento de alimentos por fotografía usando un modelo de inteligencia artificial.

ID	Requisito	Actor
RF-45	Consultar una base de datos de alimentos con buscador por nombre, alimentada por una fuente externa como Open Food Facts.	N/C
RF-46	Escanear el código de barras de un producto para obtener su información nutricional de forma automática.	C
RF-47	Identificar alimentos mediante el análisis de fotografías usando un modelo de reconocimiento de imagen basado en IA.	C
RF-48	Añadir alimentos personalizados a la base de datos (recetas propias o productos locales) con sus macronutrientes.	N
RF-49	Registrar un alimento consumido como parte del plan diario del cliente desde el propio buscador.	C
RF-50	Comparar los macronutrientes de un alimento con los objetivos nutricionales restantes del día antes de decidir si añadirlo al plan.	C

Tabla A.11: Requisitos funcionales V1 — Base de datos de alimentos.

A.2.12. Marketplace de recursos educativos (*innovación clave V1*)

El análisis de competencia reveló que algunas plataformas profesionales como Healthie incorporan un marketplace donde los nutricionistas pueden vender contenido educativo y los clientes pueden adquirirlo. La propuesta original de NutriApp

recogía esta idea y la adaptaba a sus dos paneles: el nutricionista publica sus propios recursos (guías, libros, plantillas, cursos) y gestiona sus ventas, mientras que el cliente navega el catálogo general y dispone además de una sección destacada con los recursos publicados específicamente por su propio profesional. De esta forma, la plataforma ofrecía al nutricionista una vía adicional de monetización y al cliente acceso a contenido educativo curado por alguien en quien ya confía.

ID	Requisito	Actor
RF-51	Explorar un catálogo de recursos educativos (cursos, libros, plantillas, guías) filtrado por categoría y con valoraciones de otros usuarios.	N/C
RF-52	Publicar recursos propios en el marketplace asignándoles categoría, descripción, precio y materiales asociados.	N
RF-53	Gestionar los recursos publicados y el historial de ventas propias desde la sección <i>My Published</i> .	N
RF-54	Adquirir recursos del marketplace y acceder a ellos desde una biblioteca personal (<i>My Library</i>).	C
RF-55	Visualizar una sección destacada con los recursos publicados específicamente por el propio nutricionista del cliente.	C

Tabla A.12: Requisitos funcionales V1 — Marketplace de recursos educativos.

A.2.13. Automatización de procesos (*innovación clave V1*)

La automatización de tareas repetitivas es la forma más directa de reducir la carga administrativa del nutricionista. La propuesta original ampliaba el concepto más allá del envío programado de formularios: ofrecía un motor de reglas configurables con disparadores y acciones asociadas, de forma que el profesional podía modelar procedimientos típicos de su práctica (bienvenida a nuevos clientes, seguimiento tras sesión, detección de baja adherencia, celebración de hitos) y dejar que el sistema los ejecutase sin intervención manual.

ID	Requisito	Actor
RF-41	Automatizar el envío de formularios programados (semanal, mensual, tras cada cita).	S
RF-42	Automatizar las notificaciones de nuevos mensajes, formularios asignados y eventos relevantes del sistema.	S
RF-56	Definir reglas de automatización con disparadores (nuevo cliente, baja adherencia sostenida, cita próxima, hito alcanzado) y acciones asociadas (enviar mensaje, asignar formulario, crear tarea interna).	N
RF-57	Activar, desactivar, editar o eliminar reglas de automatización de forma individual.	N
RF-58	Ejecutar automáticamente las acciones de las reglas activas cuando se cumplen sus disparadores.	S

Tabla A.13: Requisitos funcionales V1 — Automatización de procesos.

A.3. Dependencias funcionales (V1)

Los requisitos funcionales no actúan de forma aislada: muchos de ellos dependen de otros para conformar las funcionalidades completas de la plataforma. La tabla A.14 muestra estas relaciones de dependencia tal y como se formularon originalmente, permitiendo trazar cada funcionalidad hasta los requisitos que la hacen posible.

Funcionalidad	Requisitos involucrados
Gestión integral de clientes	RF-01, RF-02, RF-03, RF-04, RF-05
Ficha detallada del cliente	RF-01, RF-03, RF-05, RF-19, RF-34, RF-60
Calendario multi-clínica	RF-06, RF-07, RF-08, RF-09, RF-10, RF-11
Mensajería profesional	RF-12, RF-13
Videollamadas y llamadas de voz	RF-12, RF-43, RF-44
Comunidad entre clientes	RF-12, RF-61
Formularios personalizables	RF-14, RF-15, RF-16, RF-17
Sistema de alertas inteligentes*	RF-14, RF-18, RF-19, RF-20, RF-21, RF-22, RF-23
Formularios con reglas de alerta*	RF-14, RF-15, RF-16, RF-18, RF-19
Planes nutricionales	RF-24, RF-25, RF-26, RF-27, RF-28, RF-29
Sustitución de comidas por equivalencias	RF-27, RF-59
Lista de compra inteligente*	RF-24, RF-25, RF-27, RF-30, RF-31, RF-32
Base de datos de alimentos con IA*	RF-25, RF-45, RF-46, RF-47, RF-48, RF-49, RF-50
Check-in diario gamificado*	RF-33, RF-34, RF-35, RF-37
Seguimiento y progreso	RF-04, RF-05, RF-33, RF-36, RF-37, RF-38
Informes y análisis	RF-05, RF-34, RF-36, RF-39, RF-40
Marketplace de recursos*	RF-51, RF-52, RF-53, RF-54, RF-55
Motor de automatizaciones*	RF-18, RF-41, RF-42, RF-56, RF-57, RF-58

Tabla A.14: Dependencias funcionales V1: funcionalidades y sus requisitos. Las funcionalidades marcadas con * son innovaciones clave del planteamiento original.

En el planteamiento original se destacaba que la combinación de **formularios personalizables con reglas de alerta asociadas** constituye una innovación en sí misma: la posibilidad de que el nutricionista defina, sobre cada formulario que crea, reglas condicionales que generen alertas automáticas al ser evaluadas. Ningún competidor analizado conecta estas dos funcionalidades de forma integrada. Esta combinación emergía de la intersección de los bloques de Formularios (RF-14 a RF-16) y Alertas (RF-18, RF-19), y era la que daba al sistema su capacidad de detección proactiva de problemas.

También se observaba que el bloque de **seguimiento y progreso** actuaba como punto de convergencia de varios sistemas: los datos del check-in diario (RF-33) alimentan el heat map (RF-37) y las métricas de adherencia (RF-36), mientras que las mediciones y fotos del cliente (RF-04) eran consultadas por el nutricionista (RF-05) para evaluar la evolución.

A.4. Funcionalidades resultantes (V1)

A partir de los requisitos V1 definidos y sus dependencias, las funcionalidades inicialmente propuestas para NutriApp se agrupaban en las siguientes áreas:

Gestión de clientes (RF-01 a RF-05). Perfiles completos, clasificación por categorías, filtros de búsqueda, notas privadas del nutricionista y acceso a la evolución del cliente. Sustituye la combinación de cuaderno físico + notas del móvil + plataforma de la clínica.

Calendario y citas (RF-06 a RF-11). Agenda con vistas múltiples, soporte multi-clínica con código de colores, configuración de disponibilidad, solicitud de citas por parte del cliente y recordatorios automáticos. Sustituye Google Calendar.

Mensajería profesional (RF-12, RF-13, RF-43, RF-44). Chat integrado con historial y adjuntos, vinculado al perfil de cada cliente, con botones de videollamada y llamada de voz directamente en la cabecera del chat. Sustituye tanto a WhatsApp como a las herramientas externas de videoconsulta.

Comunidad entre clientes (RF-12, RF-61). El cliente dispone de un canal de chat con otros clientes del mismo nutricionista, pensado como apoyo entre pares. El sistema garantiza el aislamiento entre consultas de profesionales distintos, por lo que un cliente nunca puede comunicarse con clientes de un nutricionista que no sea el suyo.

Ficha detallada del cliente (RF-60). El nutricionista cuenta con una vista dedicada por cada cliente que consolida en una sola pantalla la foto, los datos básicos, las notas privadas, el plan actual, la próxima sesión, las alertas recientes y el histórico de check-ins. Evita tener que saltar entre varias secciones para obtener una visión global del caso.

Formularios personalizables (RF-14 a RF-17). Editor visual con múltiples tipos de campo, biblioteca de plantillas, asignación manual o programada, y acceso del cliente a sus formularios. Sustituye Google Forms.

Sistema de alertas inteligentes (RF-18 a RF-23). *Innovación clave V1*. Reglas condicionales por formulario, evaluación automática de respuestas, alertas por inac-

tividad, dashboard de gestión con prioridades por color y acción directa desde la alerta. Sin equivalente en la competencia.

Planes nutricionales (RF-24 a RF-29, RF-59). Constructor de dietas por día y comida, base de datos de alimentos, gestión de recetas, sistema de sustituciones por equivalencia nutricional, plantillas reutilizables y vista del plan para el cliente. Incluye la posibilidad de que el cliente cambie una comida concreta por otra opción equivalente preconfigurada por el nutricionista sin romper el equilibrio de macronutrientes del día.

Base de datos de alimentos con reconocimiento inteligente (RF-45 a RF-50). *Innovación clave V1*. Base de datos accesible desde ambos paneles, alimentada por una fuente externa como Open Food Facts, con tres vías de captura: búsqueda por nombre, escaneo de código de barras y reconocimiento de alimentos por fotografía mediante un modelo de IA. El cliente puede comparar cualquier alimento con sus objetivos nutricionales restantes del día antes de añadirlo al plan, y el nutricionista puede ampliar la base con alimentos personalizados.

Lista de la compra inteligente (RF-30 a RF-32). *Innovación clave V1*. Generación automática desde el plan, sustituciones inteligentes al marcar ingredientes como no disponibles, y gestión de la lista por el cliente. Sin equivalente en la competencia.

Check-in diario gamificado (RF-33 a RF-35). *Innovación clave V1*. Registro diario de adherencia en múltiples categorías, hábitos personalizados configurados por el nutricionista, y rachas de cumplimiento con mecánica de gamificación. Sin equivalente en la competencia.

Seguimiento y progreso (RF-36 a RF-38). Gráficos de evolución, heat map de cumplimiento estilo GitHub y comparativa de fotos de progreso. Integra datos del check-in diario y las mediciones del cliente.

Informes y análisis (RF-39, RF-40). Dashboard de métricas globales para el nutricionista y exportación de informes en PDF.

Marketplace de recursos educativos (RF-51 a RF-55). *Innovación clave V1*. Catálogo de recursos formativos (cursos, libros, plantillas, guías) con filtros por categoría y valoraciones. El nutricionista puede publicar sus propios recursos y gestionar sus ventas, y el cliente dispone de una biblioteca personal con los recursos adquiridos además de una sección destacada con los contenidos publicados específicamente por su profesional.

Motor de automatizaciones (RF-41, RF-42, RF-56 a RF-58). *Innovación clave V1*. Motor de reglas configurables con disparadores (nuevo cliente, baja adherencia sostenida, cita próxima, hito alcanzado) y acciones asociadas (enviar mensaje,

asignar formulario, crear tarea interna). Permite al nutricionista modelar procedimientos recurrentes de su práctica y delegar su ejecución en el sistema, ampliando la automatización básica de envío de formularios y notificaciones.

A.5. Nota final sobre el alcance definitivo

El alcance efectivamente implementado en el TFG corresponde al subconjunto descrito en el capítulo 3 tras la reformulación recogida en la sección 3.1. Los requisitos aquí documentados que no forman parte del alcance final —*marketplace*, motor de automatizaciones avanzado, formularios personalizables, sistema de alertas inteligentes en su versión original, lista de la compra inteligente, reconocimiento de alimentos por imagen, videollamadas, chat entre clientes, entre otros— se recogen como trabajo futuro con su correspondiente justificación de posposición.

Apéndice	B
----------	----------

Título del Apéndice B

Se pueden añadir los apéndices que se consideren oportunos.

Formalización completa del motor de generación

Este anexo recoge la formalización completa del modelo del capítulo 5: los umbrales con sus valores, las 15 reglas invariantes, los 16 tipos configurables con su formulación y el mapa de cada elemento formal a su función en el código. La notación es la del capítulo: $ha_consumido[f, d, m]$ y $gramos_alimento[f, d, m]$ son la presencia y los gramos del alimento f en la comida m del día d ; $nut_{d,n}$ es la suma escalada del nutriente n en el día d ; y $scale_target(v) = round(v \cdot s \cdot 100)$ lleva un valor real a la escala entera del modelo, con $s = 10$. Siguiendo el convenio habitual, una variable booleana dentro de un sumatorio aporta 1 si la condición se cumple y 0 en caso contrario.

C.1. Umbrales y pesos del modelo

Todos los umbrales son constantes con nombre en la configuración del solver.

Constante	Valor	Papel
NUTRIENT_SCALE	10	escala entera de los valores nutricionales
MIN_GRAMS_PRESENT	10 g	suelo absoluto de presencia
MAX_ITEMS_PER_MEAL	4	tope de alimentos por comida (R2)
MAX_SAME_FOOD_PER_DAY	2	repeticiones de un alimento en el día (R6)
MIN_DISTINCT_PER_DAY	4	variedad mínima diaria (R7)
MIN_DISTINCT_PER_WEEK	10	variedad mínima semanal (R8)
MAX_APPEARANCES_PER_DAY_RATIO	0.6	tope de reparto por día de plan
FLOOR_KCAL	1200 / 1500	suelo calórico absoluto (mujeres / hombres, R3)
PROTEIN_G_PER_KG	0.8 – 2.2	rango humano de proteína en g/kg (R4)
DEFAULT_WEIGHT_KG	70	peso por defecto si falta la antropometría
FAT_MIN_PCT_ENERGY	15	mínimo de energía en grasa, en porcentaje (R5)
MIN_ITEMS_MAIN_MEAL	2	composición mínima de las principales (R12)
CONDIMENT_MAX_PER_DAY	2	tope diario de condimentos (R14)
SWEET_FRUIT_MAX_PER_MEAL	1	dulce y fruta por comida (R15)
FALLBACK_MIN_SERVING_G	20 g	mínimo de ración sin perfil
FALLBACK_MAX_SERVING_G	250 g	máximo de ración sin perfil
SOLVE_TIME_LIMIT_S	90 s	límite de tiempo de resolución
SOLVE_RELATIVE_GAP	0.02	brecha relativa de parada
SOLVE_WORKERS	8 / 2	hilos de búsqueda (desarrollo / despliegue)

Tabla C.1: Umbrales del modelo y parámetros de resolución.

Los pesos por familia de la función objetivo son $w_{\text{kcal}} = 10$, $w_{\text{protein}} = 8$, $w_{\text{carb}} = 6$, $w_{\text{fat}} = 6$, $w_{\text{no_repeat}} = 5\,000$, $w_{\text{variety}} = 5\,000$, $w_{\text{prefer}} = 4\,000$ y $w_{\text{spread}} = 4\,000$. Las familias no nutricionales llevan incorporado un factor de 1 000 que compensa la escala entera: sus términos se cuentan en unidades de 1 (una aparición, un alimento sin usar) mientras que las desviaciones nutricionales se miden en unidades escaladas, donde una kcal son 1 000 (capítulo 5). Sobre estos pesos, cada fila de restricción modula con su peso propio de 1 a 10 (sección C.6).

C.2. Variables y enlace

Dos familias de variables por cada terna alimento, día y comida, más una tercera para los alimentos que se sirven por piezas:

$$\text{ha_consumido}[f, d, m] \in \{0, 1\}, \quad \text{gramos_alimento}[f, d, m] \in \{0, 1, \dots, \max_f\}, \\ \forall f \in F, d \in D, m \in M$$

$$\begin{aligned} \text{ha_consumido}[f, d, m] = 1 &\Rightarrow \text{gramos_alimento}[f, d, m] \geq \min_f, \\ \text{ha_consumido}[f, d, m] = 0 &\Rightarrow \text{gramos_alimento}[f, d, m] = 0 \end{aligned}$$

donde los límites por aparición salen del perfil de ración del alimento, con el suelo absoluto de presencia por debajo:

$$\begin{aligned} \min_f &= \max(\text{round}(\min_serving_g(f)), \text{MIN_GRAMS_PRESENT}) \\ \max_f &= \text{round}(\max_serving_g(f)) \end{aligned}$$

Para los alimentos con gramos por unidad $\text{gpu}(f)$, la variable entera k cuantiza los gramos a medias piezas (k cuenta medias unidades: $k = 3$ son 1,5 piezas):

$$k[f, d, m] \in \left\{0, \dots, \left\lfloor \frac{2 \cdot \max_f}{\text{gpu}(f)} \right\rfloor\right\}, \quad 2 \cdot \text{gramos_alimento}[f, d, m] = k[f, d, m] \cdot \text{gpu}(f)$$

y los alimentos que solo admiten piezas enteras (huevo, yogur) usan la variante sin medias:

$$k[f, d, m] \in \left\{0, \dots, \left\lfloor \frac{\max_f}{\text{gpu}(f)} \right\rfloor\right\}, \quad \text{gramos_alimento}[f, d, m] = k[f, d, m] \cdot \text{gpu}(f)$$

La cuantización compone con el enlace: si el alimento no está presente, los gramos quedan a cero y fuerzan $k = 0$; si lo está, el perfil acota k a las piezas que caben entre $\min_f$ y $\max_f$.

C.3. Restricciones estructurales (R1 a R8)

Todas duras y aplicadas siempre.

R1. Presencia mínima por comida.

$$\sum_{f \in F} \text{ha_consumido}[f, d, m] \geq 1, \quad \forall d \in D, m \in M$$

R2. Cota de alimentos por comida.

$$\sum_{f \in F} \text{ha_consumido}[f, d, m] \leq \text{MAX_ITEMS_PER_MEAL}, \quad \forall d \in D, m \in M$$

R3. Suelo calórico diario. Con el basal de Mifflin-St Jeor sobre el peso w_c (kg), la altura h_c (cm) y la edad a_c :

$$\text{BMR}(c) = 10 w_c + 6,25 h_c - 5 a_c + \begin{cases} +5 & \text{en hombres} \\ -161 & \text{en mujeres} \end{cases}$$

$$\text{floor}(c) = \text{máx}(\text{BMR}(c), \text{FLOOR_KCAL}[\text{sexo}])$$

$$\sum_{m \in M} \sum_{f \in F} \text{kcal100}(f) \cdot \text{gramos_alimento}[f, d, m] \geq \text{scale_target}(\text{floor}(c)),$$

$$\forall d \in D$$

R4. Rango humano de proteína. Con $\text{prot_min}(c)$ y $\text{prot_max}(c)$ iguales a 0.8 y 2.2 veces el peso del cliente (o el peso por defecto si falta):

$$\text{scale_target}(\text{prot_min}(c)) \leq \text{nut}_{d,\text{prot}} \leq \text{scale_target}(\text{prot_max}(c)),$$

$$\forall d \in D$$

R5. Grasa mínima por energía. Partiendo de $\text{fat_g} \cdot 9 \geq (\text{pct}/100) \cdot \text{kcal}$ y despejando la división con la multiplicación cruzada:

$$\left(\sum_{m,f} \text{nut100}(f, \text{fat}) \cdot \text{gramos_alimento}[f, d, m] \right) \cdot 9 \cdot 100 \geq$$

$$15 \cdot \left(\sum_{m,f} \text{kcal100}(f) \cdot \text{gramos_alimento}[f, d, m] \right), \quad \forall d \in D$$

R6. No repetir el mismo alimento dentro del día.

$$\sum_{m \in M} \text{ha_consumido}[f, d, m] \leq \text{MAX_SAME_FOOD_PER_DAY},$$

$$\forall f \in F, d \in D$$

R7. Variedad mínima diaria. Con un indicador de uso diario:

$$\text{usado_dia}[f, d] = \text{máx}_{m \in M} \text{ha_consumido}[f, d, m], \quad \forall f \in F, d \in D$$

$$\sum_{f \in F} \text{usado_dia}[f, d] \geq \text{mín}(\text{MIN_DISTINCT_PER_DAY}, |F|), \quad \forall d \in D$$

R8. Variedad mínima semanal. Para cada ventana de 7 días, con un indicador de uso en la ventana:

$$\text{usado_sem}[f] = \text{máx}_{d \in \text{ventana}, m \in M} \text{ha_consumido}[f, d, m]$$

$$\sum_{f \in F} \text{usado_sem}[f] \geq \text{mín}(\text{MIN_DISTINCT_PER_WEEK}, |F|)$$

C.4. Restricciones de plausibilidad (R9 a R15)

Todas duras y sin literal de asunción: no forman parte del vocabulario que el profesional puede reducir, así que nunca aparecen en un núcleo de infactibilidad. Se usan los subconjuntos COND (alimentos con rol de condimento), DULCE (dulces), FRUTA (fruta) y, para cada alimento f , la lista blanca de franjas $\text{momentos}(f)$

derivada de sus etiquetas de momento; $c(m)$ es la franja de la comida m y MAIN el conjunto de franjas principales (desayuno, comida, cena).

R9. Perfil de ración por aparición. Es el enlace presencia-gramos de la sección C.2: presente implica $\min_f \leq \text{gramos_alimento} \leq \max_f$.

R10. Cantidades por unidades. Es la cuantización por piezas de la sección C.2.

R11. Lista blanca de franjas.

$$\begin{aligned} \text{ha_consumido}[f, d, m] &= 0, \\ \forall f : \text{momentos}(f) &\neq \emptyset, \forall d \in D, \forall m : c(m) \notin \text{momentos}(f) \end{aligned}$$

R12. Composición mínima de las comidas principales.

$$\begin{aligned} \sum_{f \in F} \text{ha_consumido}[f, d, m] &\geq \min(\text{MIN_ITEMS_MAIN_MEAL}, |F|), \\ \forall d \in D, \forall m : c(m) &\in \text{MAIN} \end{aligned}$$

R13. El condimento nunca va solo.

$$\begin{aligned} \text{ha_consumido}[f, d, m] &\leq \sum_{f' \in F \setminus \text{COND}} \text{ha_consumido}[f', d, m], \\ \forall f \in \text{COND}, d \in D, m \in M \end{aligned}$$

Caso frontera: si el catálogo visible solo tuviera condimentos, la regla dejaría el problema sin solución por construcción; en ese caso degenerado se omite y se emite un aviso.

R14. Tope diario de condimentos.

$$\begin{aligned} \sum_{f \in \text{COND}} \sum_{m \in M} \text{ha_consumido}[f, d, m] &\leq \text{CONDIMENT_MAX_PER_DAY}, \\ \forall d \in D \end{aligned}$$

R15. Dulce y fruta como complemento.

$$\begin{aligned} \sum_{f \in \text{DULCE} \cup \text{FRUTA}} \text{ha_consumido}[f, d, m] &\leq \text{SWEET_FRUIT_MAX_PER_MEAL}, \\ \forall d \in D, m \in M \end{aligned}$$

C.5. Los 16 tipos configurables

Cada tipo se da con su naturaleza por defecto y su formulación. Las variantes duras acotan el espacio de soluciones; las blandas aportan un término a la función objetivo con el patrón de exceso y defecto de la sección C.6.

meals_per_day y plan_duration_days (estructurales). Fijan los conjuntos M y D y, con ellos, cuántas variables se crean. No generan restricción optimizable; llegan por la firma de la función.

kcal_target (blanda por defecto). Acerca la energía diaria a un valor v . Como blanda, penaliza la desviación absoluta por día. Como dura:

$$\text{nut}_{d,\text{kcal}} = \text{scale_target}(v), \quad \forall d \in D$$

macro_target (blanda por defecto). Igual que el objetivo calórico pero sobre un macronutriente (proteína, hidratos o grasa); el peso de familia se elige según el macro.

nutrient_min (configurable). Mínimo diario de un nutriente. Como dura, $\text{nut}_{d,n} \geq \text{scale_target}(v)$ para todo día. Como blanda, penaliza solo el defecto, con una variable por día:

$$\text{def}_d \geq \text{scale_target}(v) - \text{nut}_{d,n}, \quad \text{def}_d \geq 0$$

nutrient_max (configurable). Máximo diario de un nutriente. Como dura:

$$\text{nut}_{d,n} \leq \text{scale_target}(v), \quad \forall d \in D$$

Como blanda, penaliza solo el exceso:

$$\text{exc}_d \geq \text{nut}_{d,n} - \text{scale_target}(v), \quad \text{exc}_d \geq 0$$

Se pondera con el peso calórico, por ser el de referencia para límites no atados a un macro concreto.

nutrient_ratio (configurable). Acota el cociente entre dos nutrientes (a sobre b) a un valor v , representado con 3 decimales. Como dura, con cota máxima:

$$\text{nut}_{d,a} \cdot 1000 \leq \text{round}(v \cdot 1000) \cdot \text{nut}_{d,b}, \quad \forall d \in D$$

y con cota mínima la desigualdad se invierte. La variante blanda no se modela: cuando llega, el motor la omite con un aviso.

forbid_food (dura por defecto). Prohíbe un alimento concreto:

$$\text{ha_consumido}[f_0, d, m] = 0, \quad \forall d \in D, m \in M$$

forbid_tag (dura por defecto). Prohíbe todos los alimentos de una etiqueta; es el mecanismo de las alergias e intolerancias:

$$\text{ha_consumido}[f, d, m] = 0, \quad \forall f \in \text{miembros}(t), d \in D, m \in M$$

prefer_food (blanda). Bonifica cada aparición del alimento, opcionalmente solo en un tipo de comida:

$$\text{bonificación} += \sum_{d \in D} \sum_{m \in M_{\text{ctx}}} \text{ha_consumido}[f_0, d, m]$$

donde M_{ctx} son las comidas del tipo indicado, o todas si no se indica.

prefer_tag (blanda). Como la anterior, bonificando cada aparición de un alimento de la etiqueta.

meal_kcal_ratio (blanda por defecto). Reparte la energía diaria entre las comidas según porcentajes deseados. Para cada comida con porcentaje objetivo pct (escalado por 10 para conservar un decimal), la desviación se linealiza con la multiplicación cruzada:

$$\text{kcal}_{d,m} \cdot 1000 - \text{round}(pct \cdot 10) \cdot \text{nut}_{d,\text{kcal}} = \text{over}_{d,m} - \text{under}_{d,m}$$

y $\text{over} + \text{under}$ entra como penalización; $\text{kcal}_{d,m}$ es la energía escalada de la comida m del día d .

max_servings_per_period (configurable). Limita las apariciones de un alimento o de una etiqueta en ventanas de window_days días. Como dura, para cada ventana W :

$$\sum_{f \in \text{obj}} \sum_{d \in W} \sum_{m \in M} \text{ha_consumido}[f, d, m] \leq \text{cap}$$

Como blanda, penaliza el exceso sobre el tope en cada ventana.

no_repeat_food y no_repeat_tag (blandas por defecto). Impiden o penalizan que el objetivo (un alimento, o cualquier alimento de una etiqueta) se repita en menos de sep días. Con la presencia por día del objetivo:

$$\text{pres}_d = \max_{f \in \text{obj}, m \in M} \text{ha_consumido}[f, d, m]$$

la versión dura exige, en cada ventana W de sep días, $\sum_{d \in W} \text{pres}_d \leq 1$; la blanda penaliza el exceso sobre 1 en cada ventana.

forbid_combination (dura por defecto). Prohíbe que dos alimentos, o un alimento y una etiqueta, coincidan en la misma comida. Con indicadores de presencia de cada grupo en la comida (a y b , máximos de cada grupo):

$$a_{d,m} + b_{d,m} \leq 1, \quad \forall d \in D, m \in M$$

C.6. Función objetivo

La función completa que se minimiza:

$$\min \left(\sum_i \text{penalización}_i - \sum_j \text{bonificación}_j + w_{\text{variety}} \cdot P_{\text{variety}} + w_{\text{spread}} \cdot P_{\text{spread}} \right)$$

Cada desviación absoluta se linealiza con dos variables no negativas de exceso y defecto:

$$\begin{aligned} \text{over} &\geq 0, & \text{under} &\geq 0, & \text{suma} - \text{objetivo} &= \text{over} - \text{under} \\ & & & & |\text{suma} - \text{objetivo}| &= \text{over} + \text{under} \end{aligned}$$

con las cotas ajustadas al catálogo: el exceso como mucho a la cota superior del nutriente menos el objetivo, y el defecto como mucho al objetivo. La cota superior de cada nutriente por comida se apoya en R2: es la suma de las `MAX_ITEMS_PER_MEAL` mayores aportaciones individuales alcanzables (la ración máxima del alimento por su densidad escalada), y la cota diaria es esa suma por el número de comidas.

El término de variedad penaliza cada alimento no usado en todo el plan, con `usado[f]` el máximo de `ha_consumido` sobre el plan entero:

$$P_{\text{variety}} = \sum_{f \in F} (1 - \text{usado}[f])$$

El término de reparto penaliza el exceso de apariciones de un alimento sobre el tope $\text{cap} = \lceil \text{MAX_APPEARANCES_PER_DAY_RATIO} \cdot |D| \rceil$:

$$\begin{aligned} \text{exceso}[f] &\geq \left(\sum_{d \in D} \sum_{m \in M} \text{ha_consumido}[f, d, m] \right) - \text{cap}, & \text{exceso}[f] &\geq 0 \\ P_{\text{spread}} &= \sum_{f \in F} \text{exceso}[f] \end{aligned}$$

El peso efectivo de cada término de restricción es el producto del peso de su familia por el peso de la fila:

$$\text{peso efectivo} = w_{\text{familia}} \cdot \text{peso de la fila}$$

Un objetivo calórico con peso de fila 7 pesa $10 \cdot 7 = 70$ por unidad escalada de desviación (70 000 por kcal); una preferencia de familia con peso 4 pesa $4\,000 \cdot 4 = 16\,000$ por aparición. En unidades naturales son magnitudes comparables, por el factor de 1 000 de la sección C.1.

C.7. Mapa de restricciones a código

Correspondencia entre cada elemento formal y su ubicación en el solver, para que el modelo sea trazable en ambos sentidos. Se citan fichero y función, no números de línea.

Elemento formal	Código
Variables de decisión y su enlace	<code>model.build_base_model</code>
Perfil de ración $\min_f, \max_f$ (R9)	<code>model.serving_bounds</code>
Cuantización por unidades (R10)	<code>model.build_base_model</code> (variable k)
Sumas escaladas $\text{nut}_{d,n}$	<code>model.build_base_model</code> (bucle de nutrientes)
Escala entera	<code>model.scale_target, model.scaled_100</code>
R1 y R2 (presencia y tope por comida)	<code>model._structural_constraints</code>
R3 (suelo calórico, Mifflin)	<code>model.daily_kcal_floor,</code> <code>model.mifflin_bmr</code>
R4 y R5 (proteína y grasa)	<code>model._structural_constraints</code>
R6, R7 y R8 (repetición y variedad)	<code>model._structural_constraints</code>
R11 a R15 (plausibilidad)	<code>model._plausibility_constraints</code>
Franjas por número de comidas	<code>loader._SLOTS_BY_COUNT,</code> <code>loader.load_meal_type_codes</code>

Tabla C.2: Mapa a código: modelo y reglas invariantes.

Elemento formal	Código
Aplicación del catálogo y asunciones	<code>catalog.apply_constraints,</code> <code>catalog.hard_enforce</code>
$\text{kcal_target} / \text{macro_target}$	<code>catalog._h_kcal_target,</code> <code>catalog._h_macro_target</code>
$\text{nutrient_min} / \text{max} / \text{ratio}$	<code>catalog._h_nutrient_min,</code> <code>_h_nutrient_max, _h_nutrient_ratio</code>
$\text{forbid_food} / \text{forbid_tag}$	<code>catalog._h_forbid_food, _h_forbid_tag</code>
$\text{prefer_food} / \text{prefer_tag}$	<code>catalog._h_prefer_food, _h_prefer_tag</code>
meal_kcal_ratio	<code>catalog._h_meal_kcal_ratio</code>
$\text{max_servings_per_period}$	<code>catalog._h_max_servings</code>
$\text{no_repeat_food} / \text{no_repeat_tag}$	<code>catalog._h_no_repeat_food,</code> <code>_h_no_repeat_tag</code>
$\text{forbid_combination}$	<code>catalog._h_forbid_combination</code>
Linealización del valor absoluto	<code>catalog._abs_dev</code>
Función objetivo (suma ponderada)	<code>objective.set_objective</code>
P_{variety} y P_{spread}	<code>objective._variety_penalty,</code> <code>objective._spread_penalty</code>
Resolución (tiempo, brecha, hilos)	<code>result.solve, config.SOLVE_WORKERS</code>
Diagnóstico de infactibilidad	<code>result._extract_infeasible</code>

Tabla C.3: Mapa a código: catálogo clínico, objetivo y resolución.

Este texto se puede encontrar en el fichero Cascaras/fin.tex. Si deseas eliminarlo, basta con comentar la línea correspondiente al final del fichero TFGTeXiS.tex.

*–¿Qué te parece desto, Sancho? – Dijo Don Quijote –
Bien podrán los encantadores quitarme la ventura,
pero el esfuerzo y el ánimo, será imposible.*

*Segunda parte del Ingenioso Caballero
Don Quijote de la Mancha
Miguel de Cervantes*

*–Buena está – dijo Sancho –; fírmela vuestra merced.
–No es menester firmarla – dijo Don Quijote–,
sino solamente poner mi rúbrica.*

*Primera parte del Ingenioso Caballero
Don Quijote de la Mancha
Miguel de Cervantes*

